

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-a58f4dd1ed23a6468.jsonl`
- SHA-256: `59788f3c579e0f56c55189045eea53d4342cabd58e27794be4b94feece105103`
- Source modified: `2026-07-03T17:35:44+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `subagents`
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`

Transcript

1. user (2026-07-03T17:33:44.688Z)

You are a code-review finder agent. Review target: PR #452 of badminton-highlight-indexer at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, branch codex/a2-golden-manifest-portability (HEAD 5a91701). Diff: git diff origin/master..HEAD. New module backend/eval/golden_manifest.py: manifest JSON loading + path rebasing via candidate-dir search + atomic write_manifest + CLI.

ANGLE ? reuse. Check whether the new module re-implements things the codebase already has; name the existing helper to use instead:

1. Path resolution/rebasing: does backend/ already have a path-resolver or workspace/root-finder (backend/utils/workspace.py, backend/utils/*, backend/config/*) that golden_manifest.repo_root() or _candidate_dirs duplicates? Is there an existing 'find repo root' helper used elsewhere?
2. Atomic JSON write: does backend/utils (or elsewhere) already have an atomic-write/json-dump helper that write_manifest duplicates (grep os.replace, .tmp patterns)?
3. Manifest loading: do calibrate_wasb.load_golden_gts / gt_loader / experiment.py already define a manifest-spec loader or schema the new load_manifest duplicates or should share?
4. Env-var list parsing (os.pathsep split): existing config helper?
5. CLI scaffolding conventions: do sibling eval CLIs share a pattern the new build_parser breaks (e.g., a shared --manifest default constant defined elsewhere that now exists in two places ? grep 'human_lovo_manifest.json' for duplicated literals; is DEFAULT_MANIFEST duplicated against another module's constant?).

Only flag concrete duplications where you can name both locations. Return up to 6 candidates as JSON [{file, line, summary, failure_scenario}] (failure_scenario = the maintenance cost). [] if none. Final message = only the JSON array.

2. assistant (2026-07-03T17:33:47.066Z)

I'll analyze the PR for code reuse issues in the new `backend/eval/golden\_manifest.py` module. Let me start by examining the diff and then searching for existing patterns in the codebase.

3. user (2026-07-03T17:33:47.965Z)

<persisted-output>

Output too large (33.2KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\tool-results\b5bjwb67y.txt

Preview (first 2KB):

```
diff --git a/backend/eval/calibrate_local.py b/backend/eval/calibrate_local.py
index 687f790..4734f38 100644
--- a/backend/eval/calibrate_local.py
+++ b/backend/eval/calibrate_local.py
@@ -24,7 +24,6 @@ where videos.json = [{"name","trajectory","fps","frame_width","labels"}, ...]
 from __future__ import annotations

import argparse
```

```

-import json
from typing import Callable, Dict, List, Tuple

from backend.eval.calibrate_wasb import load_golden_gts
@@ -34,6 +33,7 @@ from backend.eval.calibration import (
    leave_one_video_out,
    select_best,
)
+from backend.eval.golden_manifest import load_resolved_manifest
from backend.pipeline.detectors import rally_gate
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

@@ -164,8 +164,7 @@ def main() -> None:
    ap.add_argument("--iou", type=float, default=0.5)
    ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
    args = ap.parse_args()
    with open(args.manifest, encoding="utf-8") as f:
    -     specs = json.load(f)
+     specs = load_resolved_manifest(args.manifest)
    run(specs, iou=args.iou, metric=args.metric)

diff --git a/backend/eval/distill_local.py b/backend/eval/distill_local.py
index 52ac85a..a05b3ff 100644
--- a/backend/eval/distill_local.py
+++ b/backend/eval/distill_local.py
@@ -24,13 +24,13 @@ Run:
    from __future__ import annotations

    import argparse
-import json
from typing import Dict, List, Optional, Tuple

from backend.eval import windowing as _windowing
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.classifier import LogisticRegression
from backend.eval.gemini_refine import merge_overlaps
+from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.metrics import score
from backend.eval.training import label_candidates
from backend.pipeline.detectors import rally_gate
@@ -218,8 +218,7 @@ def main() -> None:
)
...
</persisted-output>

```

4. assistant (2026-07-03T17:33:49.698Z)

Let me read the full diff to see the new golden_manifest.py file:

5. user (2026-07-03T17:33:51.522Z)

```

1      diff --git a/backend/eval/calibrate_local.py b/backend/eval/calibrate_local.py
2      index 687f790..4734f38 100644
3      --- a/backend/eval/calibrate_local.py
4      +++ b/backend/eval/calibrate_local.py
5      @@ -24,7 +24,6 @@ where videos.json =
[{"name", "trajectory", "fps", "frame_width", "labels"}, ...]
6      from __future__ import annotations
7
8      import argparse
9      -import json
10     from typing import Callable, Dict, List, Tuple
11
12     from backend.eval.calibrate_wasb import load_golden_gts

```

```

13     @@ -34,6 +33,7 @@ from backend.eval.calibration import (
14         leave_one_video_out,
15         select_best,
16     )
17 +from backend.eval.golden_manifest import load_resolved_manifest
18     from backend.pipeline.detectors import rally_gate
19     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
20
21     @@ -164,8 +164,7 @@ def main() -> None:
22         ap.add_argument("--iou", type=float, default=0.5)
23         ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
24         args = ap.parse_args()
25     -    with open(args.manifest, encoding="utf-8") as f:
26     -        specs = json.load(f)
27     +    specs = load_resolved_manifest(args.manifest)
28     run(specs, iou=args.iou, metric=args.metric)
29
30
31 diff --git a/backend/eval/distill_local.py b/backend/eval/distill_local.py
32 index 52ac85a..a05b3ff 100644
33 --- a/backend/eval/distill_local.py
34 +++ b/backend/eval/distill_local.py
35 @@ -24,13 +24,13 @@ Run:
36     from __future__ import annotations
37
38     import argparse
39 -import json
40     from typing import Dict, List, Optional, Tuple
41
42     from backend.eval import windowing as _windowing
43     from backend.eval.calibrate_wasb import load_golden_gts
44     from backend.eval.classifier import LogisticRegression
45     from backend.eval.gemini_refine import merge_overlaps
46 +from backend.eval.golden_manifest import load_resolved_manifest
47     from backend.eval.metrics import score
48     from backend.eval.training import label_candidates
49     from backend.pipeline.detectors import rally_gate
50 @@ -218,8 +218,7 @@ def main() -> None:
51     )
52     ap.add_argument("--model-out", default="output/local_rally_model.json")
53     args = ap.parse_args()
54     -    with open(args.manifest, encoding="utf-8") as f:
55     -        specs = json.load(f)
56     +    specs = load_resolved_manifest(args.manifest)
57     run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
58
59
60 diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
61 index dd3f6a1..aee6d7e 100644
62 --- a/backend/eval/fusion_golden.py
63 +++ b/backend/eval/fusion_golden.py
64 @@ -25,6 +25,7 @@ from typing import Any, Dict, List, Optional
65
66     from backend.eval import distill_local
67     from backend.eval.calibrate_wasb import load_golden_gts
68 +from backend.eval.golden_manifest import load_resolved_manifest
69     from backend.eval.training import label_candidates
70     from backend.pipeline.detectors import fusion_features as ff
71     from backend.pipeline.detectors import rally_gate
72 @@ -160,8 +161,7 @@ def run(
73     fresh: bool = False,
74     smallest_first: bool = False,
75 ) -> List[str]:
76     -    with open(manifest_path, encoding="utf-8") as f:
77     -        specs = json.load(f)

```

```

78     + specs = load_resolved_manifest(manifest_path)
79     if smallest_first:
80         # Process small videos first for quick feedback (the slow full match lands
last);
81         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
82     diff --git a/backend/eval/golden_manifest.py b/backend/eval/golden_manifest.py
83     new file mode 100644
84     index 00000000..7a01677
85     --- /dev/null
86     +++ b/backend/eval/golden_manifest.py
87     @@ -0,0 +1,378 @@
88     +"""Portable golden-corpus manifest loader and smoke checker.
89     +
90     +The owner-side golden manifest is intentionally gitignored and has historically
91     +carried absolute paths from whichever box ingested a clip. This module keeps that
92     +artifact usable after the workspace moves by resolving stale paths against the
93     +current repo layout and failing loudly when required corpus files are absent.
94     +
95     +Default local layout supported:
96     + - ../Annotation Setup/Trajectories/<trajectory>.csv
97     + - ../Annotation Setup/Golden Labelled/<labels>.csv
98     + - output/<labels>.csv
99     +
100    +Extra roots can be supplied with ``--root`` or ``RALLY_GOLDEN_SEARCH_ROOTS``.
101    +"""
102    +
103    +from __future__ import annotations
104    +
105    +import argparse
106    +import json
107    +import os
108    +import sys
109    +from dataclasses import dataclass
110    +from pathlib import Path, PureWindowsPath
111    +from typing import Any, Iterable, Sequence
112    +
113    +DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
114    +SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"
115    +DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
116    +
117    +_FIELD_DIR = {
118    +     "trajectory": "Trajectories",
119    +     "labels": "Golden Labelled",
120    + }
121    +
122    +
123    +@dataclass(frozen=True)
124    +class ManifestIssue:
125    +     """One manifest path problem found by the resolver."""
126    +
127    +     name: str
128    +     field: str
129    +     path: str
130    +     reason: str
131    +
132    +     def message(self) -> str:
133    +         shown = f" {self.path!r}" if self.path else ""
134    +         return f"{self.name}: {self.field}{shown} - {self.reason}"
135    +
136    +
137    +class ManifestPathError(RuntimeError):
138    +     """Raised when a strict manifest load finds missing required files."""
139    +
140    +     def __init__(self, manifest_path: str | os.PathLike[str], issues:

```

```

Sequence[ManifestIssue]):
141 +         self.manifest_path = os.fspath(manifest_path)
142 +         self.issues = list(issues)
143 +         detail = "\n".join(f" - {i.message()}" for i in self.issues[:12])
144 +         more = "" if len(self.issues) <= 12 else f"\n ... {len(self.issues) - 12}
more"
145 +         super().__init__(
146 +             f"golden manifest path check failed for {self.manifest_path}: "
147 +             f"{len(self.issues)} issue(s)\n{detail}{more}"
148 +         )
149 +
150 +
151 +def repo_root() -> Path:
152 +     """Return the repository root for this module."""
153 +
154 +     return Path(__file__).resolve().parents[2]
155 +
156 +
157 +def load_manifest(manifest_path: str | os.PathLike[str]) -> list[dict[str, Any]]:
158 +     """Load the flat golden manifest list and validate its outer shape."""
159 +
160 +     path = Path(manifest_path)
161 +     with path.open(encoding="utf-8") as f:
162 +         data = json.load(f)
163 +     if not isinstance(data, list):
164 +         raise ValueError(f"golden manifest must be a JSON list: {path}")
165 +     out: list[dict[str, Any]] = []
166 +     for i, row in enumerate(data):
167 +         if not isinstance(row, dict):
168 +             raise ValueError(f"golden manifest row {i} is not an object: {path}")
169 +         out.append(dict(row))
170 +     return out
171 +
172 +
173 +def resolve_manifest_paths(
174 +     manifest_path: str | os.PathLike[str],
175 +     *,
176 +     repo_root_path: str | os.PathLike[str] | None = None,
177 +     extra_roots: Iterable[str | os.PathLike[str]] = (),
178 +     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
179 +) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
180 +     """Load and rebase required path fields in a golden manifest.
181 +
182 +     Returns ``(resolved_specs, issues)``. The returned specs preserve all manifest
183 +     metadata but replace any resolved ``trajectory`` / ``labels`` path with the
184 +     existing local path. Missing required fields are reported in ``issues``.
185 +
186 +     """
187 +     manifest = load_manifest(manifest_path)
188 +     return resolve_manifest_specs(
189 +         manifest,
190 +         manifest_path,
191 +         repo_root_path=repo_root_path,
192 +         extra_roots=extra_roots,
193 +         required_fields=required_fields,
194 +     )
195 +
196 +
197 +def load_resolved_manifest(
198 +     manifest_path: str | os.PathLike[str],
199 +     *,
200 +     repo_root_path: str | os.PathLike[str] | None = None,
201 +     extra_roots: Iterable[str | os.PathLike[str]] = (),
202 +     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
203 +     strict: bool = True,

```

```

204 +) -> list[dict[str, Any]]:
205 +     """Load a manifest with stale paths rebased; optionally raise on missing files."""
206 +
207 +     specs, issues = resolve_manifest_paths(
208 +         manifest_path,
209 +         repo_root_path=repo_root_path,
210 +         extra_roots=extra_roots,
211 +         required_fields=required_fields,
212 +     )
213 +     if strict and issues:
214 +         raise ManifestPathError(manifest_path, issues)
215 +     return specs
216 +
217 +
218 +def resolve_manifest_specs(
219 +    specs: Sequence[dict[str, Any]],
220 +    manifest_path: str | os.PathLike[str],
221 +    *,
222 +    repo_root_path: str | os.PathLike[str] | None = None,
223 +    extra_roots: Iterable[str | os.PathLike[str]] = (),
224 +    required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
225 +) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
226 +    """Resolve an already-loaded manifest list."""
227 +
228 +    manifest = Path(manifest_path)
229 +    root = Path(repo_root_path).resolve() if repo_root_path else repo_root()
230 +    roots = [Path(p) for p in _env_roots()]
231 +    roots.extend(Path(p) for p in extra_roots)
232 +    required = tuple(required_fields)
233 +
234 +    resolved: list[dict[str, Any]] = []
235 +    issues: list[ManifestIssue] = []
236 +    for idx, spec in enumerate(specs):
237 +        row = dict(spec)
238 +        name = str(row.get("name") or f"row[{idx}]")
239 +        for field in required:
240 +            raw = row.get(field)
241 +            if not raw:
242 +                issues.append(ManifestIssue(name, field, "", "required field missing"))
243 +                continue
244 +            hit = resolve_existing_path(
245 +                str(raw),
246 +                field,
247 +                manifest_path=manifest,
248 +                repo_root_path=root,
249 +                extra_roots=roots,
250 +            )
251 +            if hit is None:
252 +                issues.append(ManifestIssue(name, field, str(raw), "file not found"))
253 +            else:
254 +                row[field] = str(hit)
255 +            resolved.append(row)
256 +    return resolved, issues
257 +
258 +
259 +def resolve_existing_path(
260 +    raw_path: str,
261 +    field: str,
262 +    *,
263 +    manifest_path: str | os.PathLike[str],
264 +    repo_root_path: str | os.PathLike[str],
265 +    extra_roots: Iterable[str | os.PathLike[str]] = (),
266 +) -> Path | None:
267 +    """Resolve one manifest path field to an existing file, if possible."""
268 +

```

```

269 +     for candidate in _candidate_paths(
270 +         raw_path,
271 +         field,
272 +         Path(manifest_path),
273 +         Path(repo_root_path),
274 +         [Path(p) for p in extra_roots],
275 +     ):
276 +         if candidate.is_file():
277 +             return candidate.resolve()
278 +     return None
279 +
280 +
281 +def _env_roots() -> list[Path]:
282 +     raw = os.environ.get(SEARCH_ROOTS_ENV, "")
283 +     if not raw:
284 +         return []
285 +     return [Path(p) for p in raw.split(os.pathsep) if p.strip()]
286 +
287 +
288 +def _basename(raw_path: str) -> str:
289 +     # PureWindowsPath handles stale Windows paths even when tests run on POSIX.
290 +     win_name = PureWindowsPath(raw_path).name
291 +     return win_name or Path(raw_path).name
292 +
293 +
294 +def _candidate_paths(
295 +     raw_path: str,
296 +     field: str,
297 +     manifest_path: Path,
298 +     root: Path,
299 +     extra_roots: Sequence[Path],
300 +) -> list[Path]:
301 +     basename = _basename(raw_path)
302 +     raw = Path(os.path.expandvars(os.path.expanduser(raw_path)))
303 +     manifest_dir = manifest_path.resolve().parent
304 +     candidates: list[Path] = [raw]
305 +
306 +     if not raw.is_absolute():
307 +         candidates.extend([manifest_dir / raw, root / raw])
308 +
309 +     for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
310 +         candidates.append(directory / basename)
311 +     return _dedup_paths(candidates)
312 +
313 +
314 +def _candidate_dirs(
315 +     field: str, manifest_dir: Path, root: Path, extra_roots: Sequence[Path]
316 +) -> list[Path]:
317 +     field_dir = _FIELD_DIR.get(field)
318 +     dirs: list[Path] = [manifest_dir]
319 +     if field_dir:
320 +         dirs.extend(
321 +             [
322 +                 root.parent / "Annotation Setup" / field_dir,
323 +                 root.parent / "Annotation Setup" / "Collect" / field_dir,
324 +                 root / "output",
325 +             ]
326 +         )
327 +     dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
328 +     return _dedup_paths(dirs)
329 +
330 +
331 +def _expand_root(root: Path, field: str) -> list[Path]:
332 +     field_dir = _FIELD_DIR.get(field)
333 +     dirs = [root, root / "output"]

```

```

334 +     if field_dir:
335 +         dirs.extend(
336 +             [
337 +                 root / field_dir,
338 +                 root / "Annotation Setup" / field_dir,
339 +                 root / "Annotation Setup" / "Collect" / field_dir,
340 +             ]
341 +         )
342 +     return dirs
343 +
344 +
345 +def _dedup_paths(paths: Iterable[Path]) -> list[Path]:
346 +    out: list[Path] = []
347 +    seen: set[str] = set()
348 +    for path in paths:
349 +        try:
350 +            key = os.path.normcase(os.path.abspath(os.fspath(path)))
351 +        except OSError:
352 +            key = os.fspath(path)
353 +        if key in seen:
354 +            continue
355 +        seen.add(key)
356 +        out.append(path)
357 +    return out
358 +
359 +
360 +def write_manifest(path: str | os.PathLike[str], specs: Sequence[dict[str, Any]]) ->
None:
361 +    """Write a resolved manifest JSON atomically enough for local tooling."""
362 +
363 +    out = Path(path)
364 +    out.parent.mkdir(parents=True, exist_ok=True)
365 +    tmp = out.with_name(out.name + ".tmp")
366 +    with tmp.open("w", encoding="utf-8") as f:
367 +        json.dump(list(specs), f, indent=2)
368 +        f.write("\n")
369 +    os.replace(tmp, out)
370 +
371 +
372 +def build_parser() -> argparse.ArgumentParser:
373 +    p = argparse.ArgumentParser(
374 +        description="Check/rebase the local golden corpus manifest paths."
375 +    )
376 +    p.add_argument("--manifest", default=DEFAULT_MANIFEST)
377 +    p.add_argument(
378 +        "--repo-root",
379 +        default=None,
380 +        help="repo root used for default sibling output/Annotation Setup lookups",
381 +    )
382 +    p.add_argument(
383 +        "--root",
384 +        action="append",
385 +        default=[],
386 +        help=(
387 +            "additional search root; repeatable. Also accepts roots via "
388 +            f"{{SEARCH_ROOTS_ENV}} separated by os.pathsep."
389 +        ),
390 +    )
391 +    p.add_argument(
392 +        "--require",
393 +        action="append",
394 +        choices=sorted(_FIELD_DIR),
395 +        default=None,
396 +        help="path field to require; repeatable. Defaults to trajectory and labels.",
397 +    )

```



```

398 +     p.add_argument(
399 +         "--min-count",
400 +         type=int,
401 +         default=None,
402 +         help="fail unless at least this many manifest rows are present",
403 +     )
404 +     p.add_argument(
405 +         "--write",
406 +         default=None,
407 +         help="optional path to write the resolved local manifest JSON",
408 +     )
409 +     return p
410 +
411 +
412 +def main(argv: Sequence[str] | None = None) -> int:
413 +     args = build_parser().parse_args(argv)
414 +     required = tuple(args.require or DEFAULT_REQUIRED_FIELDS)
415 +     try:
416 +         original = load_manifest(args.manifest)
417 +         resolved, issues = resolve_manifest_specs(
418 +             original,
419 +             args.manifest,
420 +             repo_root_path=args.repo_root,
421 +             extra_roots=args.root,
422 +             required_fields=required,
423 +         )
424 +     except (OSError, ValueError, json.JSONDecodeError) as e:
425 +         print(f"ERROR: {e}", file=sys.stderr)
426 +         return 2
427 +
428 +     if args.min_count is not None and len(resolved) < args.min_count:
429 +         issues.append(
430 +             ManifestIssue(
431 +                 "manifest",
432 +                 "rows",
433 +                 str(len(resolved)),
434 +                 f"expected at least {args.min_count} row(s)",
435 +             )
436 +         )
437 +
438 +     if issues:
439 +         print(
440 +             f"ERROR: golden manifest path check failed ({len(issues)} issue(s)):",
441 +             file=sys.stderr,
442 +         )
443 +         for issue in issues:
444 +             print(f"  - {issue.message()}", file=sys.stderr)
445 +         return 2
446 +
447 +     rebased = sum(
448 +         1
449 +         for before, after in zip(original, resolved)
450 +         for field in required
451 +         if str(before.get(field, "")) != str(after.get(field, ""))
452 +     )
453 +     print(
454 +         f"OK: {args.manifest} has {len(resolved)} video(s); "
455 +         f"{len(resolved) * len(required)} required path(s) present; "
456 +         f"{rebased} path(s) rebased."
457 +     )
458 +     if args.write:
459 +         write_manifest(args.write, resolved)
460 +         print(f"wrote resolved manifest: {args.write}")
461 +     return 0
462 +

```

```

463 +
464 +if __name__ == "__main__":
465 +    raise SystemExit(main())
466 diff --git a/backend/eval/rally_seg_eval.py b/backend/eval/rally_seg_eval.py
467 index 0acca2d..3846ce0 100644
468 --- a/backend/eval/rally_seg_eval.py
469 +++ b/backend/eval/rally_seg_eval.py
470 @@ -27,6 +27,7 @@ import os
471     from typing import Any, Dict, List, Optional, Tuple
472
473     from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
474 +from backend.eval.golden_manifest import load_resolved_manifest
475     from backend.eval.metrics import Interval
476     from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
477
478 @@ -185,8 +186,7 @@ def load_golden(
479     ) -> List[Dict[str, Any]]:
480         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
481         intervals (`<name>_golden_features.json` `gts`). Videos with no GTs are kept
but flagged."""
482     - with open(manifest_path) as fh:
483     -     manifest = json.load(fh)
484     + manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
485     out: List[Dict[str, Any]] = []
486     for v in manifest:
487         feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
488 diff --git a/backend/eval/rally_seq_proto.py b/backend/eval/rally_seq_proto.py
489 index 2ea9483..b017f94 100644
490 --- a/backend/eval/rally_seq_proto.py
491 +++ b/backend/eval/rally_seq_proto.py
492 @@ -29,7 +29,6 @@ Run:
493     from __future__ import annotations
494
495     import argparse
496 -import json
497     from typing import Dict, List, Tuple
498
499     import numpy as np
500 @@ -38,6 +37,7 @@ from scipy.stats import rankdata
501
502     from backend.eval.calibrate_wasb import load_golden_gts
503     from backend.eval.classifier import LogisticRegression
504 +from backend.eval.golden_manifest import load_resolved_manifest
505     from backend.eval.metrics import score
506
507     # Feature extraction is single-sourced in backend/features (ADR-008): training and
508 @@ -196,8 +196,7 @@ def main() -> None:
509         help="JSON: [{name,trajectory,fps,frame_width,labels}, ...]",
510     )
511     args = ap.parse_args()
512     - with open(args.manifest, encoding="utf-8") as f:
513     -     run(json.load(f))
514     + run(load_resolved_manifest(args.manifest))
515
516
517     if __name__ == "__main__":
518 diff --git a/backend/eval/serve_contrast.py b/backend/eval/serve_contrast.py
519 index 328857b..0f1b1f4 100644
520 --- a/backend/eval/serve_contrast.py
521 +++ b/backend/eval/serve_contrast.py
522 @@ -32,6 +32,7 @@ from typing import Any, Dict, List, Sequence, Tuple
523     from backend.eval import windowing as W
524     from backend.eval.calibrate_wasb import load_golden_gts
525     from backend.eval.gemini_refine import merge_overlaps

```

```

526 +from backend.eval.golden_manifest import load_resolved_manifest
527     from backend.eval.metrics import score
528     from backend.eval.promotion import promote_if_served_safe
529     from backend.pipeline.detectors import rally_gate
530 @@ -194,8 +195,7 @@ def main() -> None:
531         )
532         args = ap.parse_args()
533
534     - with open(args.manifest, encoding="utf-8") as f:
535     -     specs = json.load(f)
536     + specs = load_resolved_manifest(args.manifest)
537     result = run_contrast(
538         specs,
539         iou=args.iou,
540 diff --git a/backend/eval/served_gate_a.py b/backend/eval/served_gate_a.py
541 index 994b9c1..c2151fb 100644
542 --- a/backend/eval/served_gate_a.py
543 +++ b/backend/eval/served_gate_a.py
544 @@ -23,13 +23,13 @@ Run (owner box, golden trajectories present):
545     from __future__ import annotations
546
547     import argparse
548 -import json
549     from dataclasses import dataclass
550     from typing import Any, Dict, List, Optional, Tuple
551
552     from backend.config.models import FusionConfig, IndexingConfig, WasbIndexConfig
553     from backend.eval import segmentation_metrics as _segm
554     from backend.eval.calibrate_wasb import load_golden_gts
555 +from backend.eval.golden_manifest import load_resolved_manifest
556     from backend.eval.metrics import Interval, Score, score
557     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
558     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
559 @@ -196,8 +196,7 @@ def format_table(
560     def run(
561         manifest_path: str, iou: float = 0.5
562     ) -> Tuple[List[ServedVideoResult], Dict[str, Any]]:
563     - with open(manifest_path, encoding="utf-8") as f:
564     -     specs = json.load(f)
565     + specs = load_resolved_manifest(manifest_path)
566     results = evaluate_manifest(specs, iou=iou)
567     return results, aggregate(results)
568
569 diff --git a/docs/ALPHA_LAUNCH_READINESS.md b/docs/ALPHA_LAUNCH_READINESS.md
570 index 04d46d2..bcc7fea 100644
571 --- a/docs/ALPHA_LAUNCH_READINESS.md
572 +++ b/docs/ALPHA_LAUNCH_READINESS.md
573 @@ -96,7 +96,7 @@ Legend: TODO = not started, IN PROGRESS = actively being worked, DONE
= shipped/
574 |----|-----|-----|-----|-----|-----|----|
575 | A0 | Create this alpha readiness tracker and index it in docs | `badminton-highlight-
indexer` | - | No | DONE | #451 |
576 | A1 | Stand up alpha serving endpoint: Cloud Run/GPU or approved host, health check,
edge auth ON, upload -> process -> jobs -> compile -> download verified | `khelsutra` +
`badminton-highlight-indexer` | A0 | Owner spend / CF config | TODO | - |
577 -| A2 | Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke
check that fails loudly when labels or trajectories are missing | `badminton-highlight-indexer`
| A0 | No | TODO | - |
578 +| A2 | Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke
check that fails loudly when labels or trajectories are missing | `badminton-highlight-indexer`
| A0 | No | IN PROGRESS | this PR |
579 | A3 | Promote the 15-video source-video/MD5 records into the collector/vault path;
reconcile collector's six-video source manifest with the 15-video eval corpus | `sports-data-
collector` + vault | A2 | Owner upload/storage scope | TODO | - |
580 | A4 | Refresh the n=15 nightly regression baseline intentionally and record the exact

```

```

command/output; keep stale-baseline warnings until reviewed | `badminton-highlight-indexer` | A2
| Reviewer sign-off | TODO | - |
581 | A5 | Recompute the heuristic n=15 floor and replace the stale `heuristic_lovo_n6`
floor for future Gen-0 comparisons | `badminton-highlight-indexer` | A2 | No | TODO | - |
582 @@ -118,6 +118,15 @@ Legend: TODO = not started, IN PROGRESS = actively being worked,
DONE = shipped/
583 5. **Ablation/fusion PRs:** fix ablation import, complete 15-video features, rerun
promotion/rejection decisions.
584 6. **Launch decision PR/docs update:** record ship-gate target, C3 status, alpha copy
limits, and launch checklist.
585
586 +A2 smoke command:
587 +
588 +```powershell
589 +python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json
--min-count 15
590 +python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json
--min-count 15 --write output/human_lovo_manifest.local.json
591 +```
592 +
593 +The checker resolves stale absolute paths against the repo's sibling `Annotation Setup`
folders and `output/`, then exits non-zero if any required `trajectory` or `labels` file is
absent.
594 +
595 ## 9. Open questions
596
597 1. **Owner:** Is the first alpha allowed to use manual/offline corpus promotion, or
should P10/P11 be reopened before any tester wave?
598 diff --git a/docs/data_pipeline/GOLDEN_VIDEOS.md b/docs/data_pipeline/GOLDEN_VIDEOS.md
599 index 6d68eeb..bb197bd 100644
600 --- a/docs/data_pipeline/GOLDEN_VIDEOS.md
601 +++ b/docs/data_pipeline/GOLDEN_VIDEOS.md
602 @@ -22,6 +22,11 @@ auditable. Updated whenever a new video is labeled + ingested.
603 - **Ingestion** = copy the trajectory + labels to durable paths, write
`output/<name>_golden_features.json`
604 (`gts` + candidates), append a row to `output/human_lovo_manifest.json`.
(`scratch/at_par.py` scores a
605 single video vs Gemini + golden; `backend/eval/rally_seg_eval.py` runs the whole
corpus.)
606 +- **Path smoke check:** before running corpus evals, run
607 + `python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json
--min-count 15`.
608 + The checker rebases stale absolute paths against sibling `Annotation Setup` folders
and `output/`, and
609 + exits non-zero if any required trajectory or labels file is missing. Add `--write
output/human_lovo_manifest.local.json`
610 + when a resolved local manifest artifact is useful.
611 - **Guardrails:** owned footage only; **minors excluded**; labels never derived from a
paid LLM (Gemini is
612 inference/reference only). Commercial-clean.
613
614 @@ -57,4 +62,5 @@ auditable. Updated whenever a new video is labeled + ingested.
615 ## How to isolate the golden videos
616 Each row's **source proxy** column is the file to copy into a dedicated golden-videos
location. The labels
617 live beside the proxy (`<name>.rallies.csv`) and a durable copy is in
`output/<name>.rallies.csv`; the
618 -trajectory is in `Collect/Trajectories/` (path in the manifest). Keep this table in
sync on every new ingest.
619 +trajectory is in `Annotation Setup/Trajectories/` (older manifests may still say
`Collect/Trajectories/`;
620 +use the path smoke checker above). Keep this table in sync on every new ingest.
621 diff --git a/tests/test_fusion_compare.py b/tests/test_fusion_compare.py
622 index d3477c8..f042eea 100644
623 --- a/tests/test_fusion_compare.py

```

```

624     +++ b/tests/test_fusion_compare.py
625     @@ -203,6 +203,8 @@ def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
626
627         out_dir = str(tmp_path / "out")
628         os.makedirs(out_dir)
629         + for name in ("a.csv", "b.csv", "a.rallies.csv", "b.rallies.csv"):
630         +     (tmp_path / name).write_text("x\n", encoding="utf-8")
631         manifest = [
632             {
633                 "name": "alpha",
634     @@ -267,6 +269,8 @@ def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
635         os.makedirs(out_dir)
636         (tmp_path / "big.csv").write_text("x" * 2000)
637         (tmp_path / "small.csv").write_text("x" * 20)
638         + (tmp_path / "b.rallies.csv").write_text("x\n", encoding="utf-8")
639         + (tmp_path / "s.rallies.csv").write_text("x\n", encoding="utf-8")
640         manifest = [
641             {
642                 "name": "big",
643     diff --git a/tests/test_fusion_golden_telemetry.py
b/tests/test_fusion_golden_telemetry.py
644     index 1fab90..bdefc88 100644
645     --- a/tests/test_fusion_golden_telemetry.py
646     +++ b/tests/test_fusion_golden_telemetry.py
647     @@ -45,6 +45,9 @@ def _patch_telemetry(monkeypatch, gpu_fn=_fake_gpu, sys_fn=_fake_sys):
648
649
650     def _write_manifest(tmp_path, names):
651         + for n in names:
652         +     (tmp_path / f"{n}.csv").write_text("x\n", encoding="utf-8")
653         +     (tmp_path / f"{n}.rallies.csv").write_text("x\n", encoding="utf-8")
654         manifest = [
655             {
656                 "name": n,
657     diff --git a/tests/test_golden_manifest.py b/tests/test_golden_manifest.py
658     new file mode 100644
659     index 0000000..59ce1e5
660     --- /dev/null
661     +++ b/tests/test_golden_manifest.py
662     @@ -0,0 +1,158 @@
663     +import json
664     +from pathlib import Path
665     +
666     +import pytest
667     +
668     +from backend.eval import golden_manifest as gm
669     +
670     +
671     +def _write(path: Path, body: str = "x\n") -> Path:
672     +    path.parent.mkdir(parents=True, exist_ok=True)
673     +    path.write_text(body, encoding="utf-8")
674     +    return path
675     +
676     +
677     +def _write_manifest(repo: Path, rows: list[dict]) -> Path:
678     +    path = repo / "output" / "human_lovo_manifest.json"
679     +    path.parent.mkdir(parents=True, exist_ok=True)
680     +    path.write_text(json.dumps(rows), encoding="utf-8")
681     +    return path
682     +
683     +
684     +def test_resolves_stale_absolute_paths_to_workspace_layout(tmp_path):
685     +    workspace = tmp_path / "workspace"
686     +    repo = workspace / "badminton-highlight-indexer"
687     +    repo.mkdir(parents=True)

```

```

688 +     traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo_traj.csv")
689 +     old_label = _write(
690 +         workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
691 +     )
692 +     output_label = _write(repo / "output" / "Bar.rallies.csv")
693 +     manifest = _write_manifest(
694 +         repo,
695 +         [
696 +             {
697 +                 "name": "foo",
698 +                 "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
699 +                 "labels": "C:/old/Annotation Setup/Collect/Golden
Labelled/Foo.rallies.csv",
700 +                 "fps": 30,
701 +                 "frame_width": 1920,
702 +             },
703 +             {
704 +                 "name": "bar",
705 +                 "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
706 +                 "labels": "C:/old/badminton-highlight-indexer/output/Bar.rallies.csv",
707 +                 "fps": 30,
708 +                 "frame_width": 1920,
709 +             },
710 +         ],
711 +     )
712 +
713 +     specs, issues = gm.resolve_manifest_paths(manifest, repo_root_path=repo)
714 +
715 +     assert issues == []
716 +     assert Path(specs[0]["trajectory"]) == traj.resolve()
717 +     assert Path(specs[0]["labels"]) == old_label.resolve()
718 +     assert Path(specs[1]["labels"]) == output_label.resolve()
719 +
720 +
721 + def test_strict_loader_reports_missing_required_files(tmp_path):
722 +     repo = tmp_path / "repo"
723 +     repo.mkdir()
724 +     manifest = _write_manifest(
725 +         repo,
726 +         [
727 +             {
728 +                 "name": "missing",
729 +                 "trajectory": "C:/old/missing_traj.csv",
730 +                 "labels": "C:/old/missing.rallies.csv",
731 +             }
732 +         ],
733 +     )
734 +
735 +     with pytest.raises(gm.ManifestPathError) as excinfo:
736 +         gm.load_resolved_manifest(manifest, repo_root_path=repo)
737 +
738 +     assert len(excinfo.value.issues) == 2
739 +     assert "file not found" in str(excinfo.value)
740 +
741 +
742 + def test_trajectory_only_resolution_leaves_label_optional(tmp_path):
743 +     workspace = tmp_path / "workspace"
744 +     repo = workspace / "badminton-highlight-indexer"
745 +     repo.mkdir(parents=True)
746 +     traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
747 +     manifest = _write_manifest(
748 +         repo,
749 +         [

```

```

750 +         {
751 +             "name": "foo",
752 +             "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
753 +             "labels": "C:/old/missing.rallies.csv",
754 +         }
755 +     ],
756 + )
757 +
758 + specs, issues = gm.resolve_manifest_paths(
759 +     manifest, repo_root_path=repo, required_fields=("trajectory",)
760 + )
761 +
762 + assert issues == []
763 + assert Path(specs[0]["trajectory"]) == traj.resolve()
764 + assert specs[0]["labels"] == "C:/old/missing.rallies.csv"
765 +
766 +
767 +def test_cli_writes_resolved_manifest(tmp_path, capsys):
768 +     workspace = tmp_path / "workspace"
769 +     repo = workspace / "badminton-highlight-indexer"
770 +     repo.mkdir(parents=True)
771 +     _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
772 +     _write(repo / "output" / "Foo.rallies.csv")
773 +     manifest = _write_manifest(
774 +         repo,
775 +         [
776 +             {
777 +                 "name": "foo",
778 +                 "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
779 +                 "labels": "C:/old/badminton-highlight-indexer/output/Foo.rallies.csv",
780 +             }
781 +         ],
782 +     )
783 +     out = repo / "output" / "human_lovo_manifest.local.json"
784 +
785 +     rc = gm.main(
786 +         [
787 +             "--manifest",
788 +             str(manifest),
789 +             "--repo-root",
790 +             str(repo),
791 +             "--min-count",
792 +             "1",
793 +             "--write",
794 +             str(out),
795 +         ]
796 +     )
797 +
798 +     captured = capsys.readouterr()
799 +     assert rc == 0
800 +     assert "OK:" in captured.out
801 +     assert out.is_file()
802 +     resolved = json.loads(out.read_text(encoding="utf-8"))
803 +     assert Path(resolved[0]["trajectory"]).is_file()
804 +     assert Path(resolved[0]["labels"]).is_file()
805 +
806 +
807 +def test_cli_fails_loudly_on_missing_file(tmp_path, capsys):
808 +     repo = tmp_path / "repo"
809 +     repo.mkdir()
810 +     manifest = _write_manifest(
811 +         repo,
812 +         [{"name": "foo", "trajectory": "missing.csv", "labels":
"missing.rallies.csv"}],
813 +     )

```

```

814 +
815 + rc = gm.main(["--manifest", str(manifest), "--repo-root", str(repo)])
816 +
817 + captured = capsys.readouterr()
818 + assert rc == 2
819 + assert "golden manifest path check failed" in captured.err
820 + assert "foo: trajectory" in captured.err
821 diff --git a/tests/test_served_gate_a_regression.py
b/tests/test_served_gate_a_regression.py
822 index 7fd7630..56848df 100644
823 --- a/tests/test_served_gate_a_regression.py
824 +++ b/tests/test_served_gate_a_regression.py
825 @@ -37,7 +37,6 @@ re-measurement as the corpus grows.
826
827     from __future__ import annotations
828
829     -import json
830     import os
831
832     import pytest
833     @@ -49,6 +48,10 @@ pytest.importorskip("cv2")
834     from backend.eval import segmentation_metrics as _segm # noqa: E402
835     from backend.eval import served_gate_a as sga # noqa: E402
836     from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
837 +from backend.eval.golden_manifest import ( # noqa: E402
838 +    ManifestPathError,
839 +    load_resolved_manifest,
840 +)
841     from backend.eval.metrics import score # noqa: E402
842     from backend.pipeline.detectors import rally_gate # noqa: E402
843     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
844     @@ -64,18 +67,14 @@ MANIFEST = os.environ.get(
845
846
847     def _golden_specs():
848     -    """Return the manifest specs whose trajectory + label files are all present, else
849     []."""
850     +    """Return rebased manifest specs when trajectory + label files are present, else
851     []."""
852     if not os.path.isfile(MANIFEST):
853         return []
854     - with open(MANIFEST, encoding="utf-8") as f:
855     -     specs = json.load(f)
856     -     ok = [
857     -         s
858     -         for s in specs
859     -         if os.path.isfile(str(s.get("trajectory", "")))
860     -         and os.path.isfile(str(s.get("labels", "")))
861     -     ]
862     -     return ok if len(ok) >= 6 else []
863     + try:
864     +     specs = load_resolved_manifest(MANIFEST)
865     + except ManifestPathError:
866     +     return []
867     + return specs if len(specs) >= 6 else []
868
869     _SPECS = _golden_specs()
870 diff --git a/training/gen0/harness.py b/training/gen0/harness.py
871 index 6eec29f..433d2cb 100644
872 --- a/training/gen0/harness.py
873 +++ b/training/gen0/harness.py
874 @@ -36,6 +36,7 @@ from typing import Dict, List, Optional
875     import numpy as np

```



```

876     from backend.eval.classifier import LogisticRegression
877 +from backend.eval.golden_manifest import load_resolved_manifest
878     from backend.eval.gt_loader import load_gt_intervals
879     from backend.eval.rally_seq_proto import (
880         _seg_f1,
881     @@ -217,8 +218,7 @@ def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
882     def run(
883         manifest_path: str, out_dir: str, version: str, floor_path: Optional[str] = None
884     ) -> dict:
885     -     with open(manifest_path, encoding="utf-8") as f:
886     -         specs = json.load(f)
887     +     specs = load_resolved_manifest(manifest_path)
888         floor = None
889         if floor_path:
890             with open(floor_path, encoding="utf-8") as f:
891

```

6. assistant (2026-07-03T17:33:54.093Z)

Now let me examine the new `golden\_manifest.py` file more closely and search for existing patterns in the codebase:

7. user (2026-07-03T17:33:55.716Z)

```

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/annotations/automated_provider.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/annotations/base.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/annotations/file_provider.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/annotations/__init__.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/auth.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/job_worker.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/uploads.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/__init__.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/billing.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/base.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/cloud_run.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/__init__.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/loader.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/models.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/presets.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/startup.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/__init__.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/ablation.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/ablation_pdf.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/annotations.py

```

8. assistant (2026-07-03T17:33:57.766Z)

Let me search for existing path resolution and workspace utilities:

9. user (2026-07-03T17:33:58.726Z)

```

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/utls/annotation_proxy.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utls/app_home.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utls/ffmpeg.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utls/freespace.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utls/geometry.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utls/hashing.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/utls/logging_setup.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utls/paths.py

```

```
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utils/redact.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utils/run_telemetry.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utils/single_instance.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utils/video.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utils/workspace.py
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/utils/__init__.py
```

10. assistant (2026-07-03T17:34:00.987Z)

Perfect! Let me check the workspace and paths utilities:

11. user (2026-07-03T17:34:01.504Z)

```
1      """Per-video workspace path helper ? one folder per video, local?cloud identical.
2
3      See ``docs/PER_VIDEO_WORKSPACE.md``. The goal: every per-video artifact lives under
4      ONE folder keyed by the MD5 ``video_id``
5      (``backend/utils/hashing.py::compute_video_id``):
6
7          <root>/[<prefix>]/<video_id>/{source,detector,outputs,reels,logs,tmp}/...
8
9      The ``root`` is a **storage URI** (``local`` path / ``gcs://`` / ``s3://`` /
10     ``gdrive://``), so a
11     LOCAL deployment and a CLOUD (bucket) deployment share the *same relative layout* ? only
12     the root
13     differs. This module is **pure** (string/URI joins, no I/O) so it is trivially testable
14     and works
15     for every backend; the storage layer (``backend/storage``) does the actual read/write.
16
17     Two roots by design (D3): a **durable** workspace root (may be ``gcs://``) for artifacts
18     worth
19     keeping, and a **fast local compute-scratch** mirror (always local) for frame caches /
20     extracted
21     chunks that must never be written to a cloud root. ``tmp`` is the local-only bucket.
22
23     This is Phase 0: the helper + config fields only ? NOT yet wired into write sites (that
24     converges
25     in P2?P4, flag-gated by ``storage.single_folder_per_video``). No behaviour changes on
26     import.
27
28     """
29
30     from __future__ import annotations
31
32     import os
33     from dataclasses import dataclass
34
35     # Per-video subfolders. ``tmp`` is LOCAL-ONLY (frame PNGs, handoff/yolo/gemini chunks,
36     compile
37     # clips) ? huge + ephemeral, never pushed to a cloud workspace; cache-hygiene drops it.
38     SUBDIRS = ("source", "detector", "outputs", "reels", "logs", "tmp")
39     LOCAL_ONLY_SUBDIRS = ("tmp",)
40
41     def join_uri(root: str, *parts: str) -> str:
42         """Join path components onto a storage root with forward slashes.
43
44         Matches how the codebase already composes storage URIs (e.g.
45         ``f"{prefix}/weights/..."`` in
46         ``backend/worker``). Works for ``gcs://bucket``, ``s3://bucket``, ``local:///abs``
47         and bare
48         paths alike; empty/blank parts are skipped so ``join_uri(root, "", "x")`` ==
49         ``join_uri(root,
50         "x")``. Forward slashes are valid path separators in Python on Windows too.
```

```

39         """
40         base = root.rstrip("/")
41         # Skip empty / whitespace-only parts; for real parts strip only slashes so spaces
inside a
42         # name (e.g. a mounted "My Drive" path) are preserved.
43         tail = "/".join(p.strip("/") for p in parts if p and p.strip())
44         return f"{base}/{tail}" if tail else base
45
46
47     def resolve_workspace_root(
48         workspace_root: str, output_root: str, output_dir: str
49     ) -> str:
50         """Durable workspace root, by precedence: ``storage.workspace_root`` ?
51         ``storage.output_root`` ? ``output.output_dir``. The first non-empty wins (non-
breaking)."""
52         return (
53             (workspace_root or "").strip()
54             or (output_root or "").strip()
55             or (output_dir or "output")
56         )
57
58
59     @dataclass(frozen=True)
60     class VideoWorkspace:
61         """The single folder for one video. Pure path math; build with :func:`for_video`.
62
63         ``base`` = ``<root>/[<prefix>]/<video_id>``. Subfolder accessors return URIs/paths
under it.
64         """
65
66         root: str
67         video_id: str
68         prefix: str = "workspaces"
69
70         @property
71         def base(self) -> str:
72             if self.prefix:
73                 return join_uri(self.root, self.prefix, self.video_id)
74             return join_uri(self.root, self.video_id)
75
76         def path(self, sub: str = "", name: str = "") -> str:
77             """A path under this video's folder: ``base[/sub][/name]``."""
78             return join_uri(self.base, sub, name)
79
80         @property
81         def source(self) -> str:
82             return self.path("source")
83
84         @property
85         def detector(self) -> str:
86             return self.path("detector")
87
88         @property
89         def outputs(self) -> str:
90             return self.path("outputs")
91
92         @property
93         def reels(self) -> str:
94             return self.path("reels")
95
96         @property
97         def logs(self) -> str:
98             return self.path("logs")
99
100         @property

```

```

101         def tmp(self) -> str:
102             return self.path("tmp")
103
104
105     def for_video(root: str, video_id: str, prefix: str = "workspaces") -> Videoworkspace:
106         """Build the :class:`Videoworkspace` for ``video_id`` under ``root`` (a storage
107         URI)."""
108         if not video_id:
109             raise ValueError("video_id is required to build a per-video workspace")
110         return Videoworkspace(root=root, video_id=video_id, prefix=prefix)
111
112     def local_scratch_base(scratch_root: str, video_id: str) -> str:
113         """The fast LOCAL compute-scratch mirror for a video (always native FS, never a
114         cloud root).
115         Used for frame caches / extracted chunks (the ``tmp/`` bucket) during compute,
116         before durable
117         artifacts are pushed to the workspace. Uses ``os.path.join`` (OS-native separators)
118         since this
119         is always a local filesystem path.
120         """
121         if not video_id:
122             raise ValueError("video_id is required for a local scratch path")
123         return os.path.join(scratch_root or os.getcwd(), video_id, "tmp")

```

12. user (2026-07-03T17:34:03.990Z)

```

1     """Path-safety helpers for the API boundary (untrusted request fields)."""
2
3     import os
4     from typing import Any, Optional
5
6
7     def output_base(config: Any) -> str:
8         """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
9         (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
10         so the
11         chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
12         ``output.output_dir`` instead of
13         a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
14         literal.
15         This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
16         may be a
17         ``gs://`` URI you can't ``mkdirs``). Accepts the typed AppConfig or the legacy
18         dict-shim
19         (``.get("output", {})).get("output_dir")``); any miss or empty value falls back to
20         ``"output"``.
21         """
22         try:
23             out = (
24                 config.output.output_dir
25                 if not isinstance(config, dict)
26                 else config.get("output", {}).get("output_dir")
27             )
28         except Exception:
29             out = None
30         return out or "output"
31
32     def safe_output_filename(name: str) -> str:
33         """Validate an output filename is a BARE name (no dir part, no traversal, not
34         absolute).

```

```

30         `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
31         and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
32         primitive. Returns the name unchanged when safe; raises ValueError otherwise.
33         """
34         if not name or not name.strip():
35             raise ValueError("output_filename must not be empty")
36         if "\x00" in name:
37             raise ValueError("output_filename must not contain a null byte")
38         if "/" in name or "\\" in name:
39             raise ValueError(f"output_filename must not contain a path separator: {name!r}")
40         if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
41             raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
42         return name
43
44
45     def resolve_under_root(path: str, root: str) -> str:
46         """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
47         rp = os.path.realpath(path)
48         rr = os.path.realpath(root)
49         try:
50             contained = os.path.commonpath([rp, rr]) == rr
51         except ValueError: # different drives on Windows
52             contained = False
53         if not contained:
54             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
55         return rp
56
57
58     def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
59         """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
60
61         With no allowed_root (default, single-user local mode) only the null-byte check
applies,
62         so the local 'point it at any of my videos' workflow is preserved. Setting an
63         allowed_root (a future hosted/tenant deployment) enforces containment.
64         """
65         if "\x00" in path:
66             raise ValueError("video_path must not contain a null byte")
67         if allowed_root:
68             resolve_under_root(path, allowed_root)
69

```

13. assistant (2026-07-03T17:34:06.100Z)

Now let me search for repo_root patterns and atomic JSON writes:

14. user (2026-07-03T17:34:07.230Z)

Found 4 files
backend\eval\golden_manifest.py
backend\pipeline\stitcher\compiler.py
backend\pipeline\stitcher\watermark_i18n.py
backend\eval\golden_fixtures.py

15. user (2026-07-03T17:34:09.074Z)

backend\eval\golden_manifest.py:64: def repo_root() -> Path:

16. user (2026-07-03T17:34:11.008Z)

```

1         """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.

```

```

2
3     `docs/GOLDEN_REGRESSION_FIXTURES.md` (§4, §6, §7). This module is the **single shared
4     replay code path** used by both the freeze tool and the characterization gate, and it
5     deliberately stays on the **CONTROL path** (D5 in that doc): the learned
6     variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7     across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8     only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9     merges overlaps, and **avoids numpy EXECUTION on the replay path** ? i.e. no numpy array
/
10    LogisticRegression math runs while replaying. (numpy is still *imported* transitively
via
11    `experiment` ? the learned-variant classifier module and MUST be installed; the CONTROL
12    path simply never calls into it.)
13
14    The replay reuses the already-shipped eval stack VERBATIM::
15
16        parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
17        ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
18        ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
19
20    Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:
21
22    * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
23    provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
24    * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true``,
``license="synthetic"`.
25    * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
26    person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
27    * The windowing params ``(vt, mg, mwd)`` are **pinned inside each fixture's provenance**
and
28    passed explicitly to ``build_candidates`` as a raw 3-tuple ? we do NOT import or rely
on the
29    named ``windowing.SERVED`` constant / ``WindowingPreset``, so a future SERVED retune
cannot
30    silently churn fixtures.
31    * ``max_win`` (W1 bounded-windowing) is **RECORDED** in provenance/manifest but NOT
applied by
32    the replay: ``distill_local.build_candidates`` (shared code, intentionally not
modified here)
33    takes only the 3-tuple ``(vt, mg, mwd)`` and
``TrackNetRunner.trajectory_to_action_windows``
34    has no max-window arg, so these fixtures characterize the **W1-OFF** path by design.
The pinned
35    ``max_win`` is a forward-looking annotation, NOT a knob the gate exercises ? do not
assume it
36    tracks a future SERVED W1 flip (it would need new wiring + a re-freeze to take
effect).
37
38    §6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
39    suite proves only that the deterministic post-trajectory transforms are unchanged for
the
40    frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
41    that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
42    field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
43
44    Offline + deterministic. The replay imports `tracknet_runner` (stdlib-only for the
45    parse/window math ? no TF) and reaches no numpy/cv2 EXECUTION on the CONTROL path, but
it is

```

```

46     NOT a zero-third-party-import module: importing `experiment` transitively imports numpy
(via
47     the learned-variant classifier), so numpy must be installed even though the CONTROL
replay
48     never calls into it.
49     """
50

```

17. user (2026-07-03T17:34:13.758Z)

```

50
51     from __future__ import annotations
52
53     import argparse
54     import json
55     import math
56     import re
57     import sys
58     from dataclasses import dataclass
59     from pathlib import Path
60     from typing import Any, Dict, List, Optional, Sequence, Tuple
61
62     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
-----
63     from backend.eval import distill_local
64     from backend.eval.experiment import CONTROL, VideoCandidates, predict
65     from backend.eval.gt_loader import load_gt_intervals
66     from backend.eval.metrics import score
67     from backend.eval.regression import gt_hash
68
69     Interval = Tuple[float, float]
70
71     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
72     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
73     CURRENT_SCHEMA = 1
74
75     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
76     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
77     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
78
79     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
80     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
81     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
82     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
83     #: re-states these in its provenance; the replay reads them from there.
84     PINNED_VT = 0.02
85     PINNED_MERGE_GAP = 2.0
86     PINNED_MIN_WINDOW = 2.0
87     #: W1 (bounded-windowing) max-window seconds. 0 = OFF, matching the SERVED default.
NOTE: this is
88     #: only RECORDED (provenance/manifest) ? it is NOT applied by the replay.
``_pinned_proposal``
89     #: returns just ``(vt, mg, mwd)`` and ``distill_local.build_candidates`` (shared code,
deliberately
90     #: NOT modified by this harness) takes no max-window arg, so the fixtures characterize
the W1-OFF
91     #: path. Do NOT assume this value tracks a future SERVED W1 flip ? that would need new
wiring.
92     PINNED_MAX_WINDOW = 0.0

```

```

93
94 #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
stated
95 #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
exactly these.
96 SHUTTLE_FEATURES: Tuple[str, ...] = (
97     "duration",
98     "peak_velocity",
99     "mean_velocity",
100     "active_ratio",
101     "net_crossings",
102 )
103 assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
104     "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
105 )
106
107 #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
108 #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
109 FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
110
111 _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
112 _FLOAT_TOL = 1e-6 # abs tolerance when parse-comparing recomputed vs committed floats
113
114 #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
115 #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
116 _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
117
118 #: Strict slug charset for a real-fixture ``--label`` ? lowercase alnum, ``_``/``-`` (no
spaces,
119 #: no punctuation, no uppercase). A free-text label ("Lin Dan vs Lee, India Open 2024")
is an
120 #: attribution back-channel that ``_scan_forbidden`` does NOT catch (it isn't a
FORBIDDEN_KEY / MD5
121 #: / source-path token), so it is rejected up-front by construction (red-team: free-text
id leak).
122 _LABEL_SLUG_RE = re.compile(r"^[a-z0-9][a-z0-9_\-]*$")
123
124
125 # =====
126 # Synthetic trajectory generation (deterministic ? NO random)
127 # =====
128
129
130 @dataclass(frozen=True)
131 class RallySpec:
132     """One in-flight rally segment: the shuttle oscillates across the net midline.
133
134     ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
135     period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
136     net midline along the play axis. Deterministic sine motion ? fixed
137     velocity/crossings.
138     """
139     start_frame: int
140     n_frames: int
141     period: float = 30.0
142     amplitude: float = 400.0
143
144
145 @dataclass(frozen=True)
146 class GapSpec:
147     """Dead-time between rallies ? what separates one window from the next.

```


148
149 `kind="still"` ? shuttle visible but resting (velocity ? 0 ? no active frames);

18. assistant (2026-07-03T17:34:15.939Z)

Now let me search for atomic write patterns and JSON dumps in the codebase:

19. user (2026-07-03T17:34:17.317Z)

Found 24 files
backend\eval\fusion_golden.py
backend\eval\serve_contrast.py
backend\eval\rally_seg_eval.py
backend\eval\golden_manifest.py
backend\api\uploads.py
backend\infrastructure\database_jobs.py
backend\infrastructure\database_media.py
backend\infrastructure\database_user_models.py
backend\main.py
backend\storage\gdrive_api.py
backend\utils\annotation_proxy.py
backend\worker__main__.py
backend\utils\run_telemetry.py
backend\pipeline\detectors\wasb_infer.py
backend\eval\per_user_regression.py
backend\eval\regression.py
backend\flywheel.py
backend\eval\classifier.py
backend\eval\experiment.py
backend\eval\fusion_audio.py
backend\eval\golden_fixtures.py
backend\eval\golden_real_fixtures.py
backend\config\loader.py
backend\eval\ablation.py

20. assistant (2026-07-03T17:34:19.092Z)

Let me check some of these for existing atomic write patterns:

21. user (2026-07-03T17:34:19.985Z)

```
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-  
indexer\backend\eval\golden_fixtures.py-568-#  
=====  
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-  
indexer\backend\eval\golden_fixtures.py-569-  
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-  
indexer\backend\eval\golden_fixtures.py-570-  
backend\eval\golden_fixtures.py:571:def _write_manifest(specs: Sequence[FixtureSpec],  
fixtures_dir: Path) -> None:  
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-  
indexer\backend\eval\golden_fixtures.py-572-     entries = [  
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-  
indexer\backend\eval\golden_fixtures.py-573-         {  
backend\eval\golden_fixtures.py-574-             "slug": spec.slug,  
--  
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-  
indexer\backend\eval\golden_manifest.py-270-     return out  
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-  
indexer\backend\eval\golden_manifest.py-271-  
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-  
indexer\backend\eval\golden_manifest.py-272-  
backend\eval\golden_manifest.py:273:def write_manifest(path: str | os.PathLike[str], specs:  
Sequence[dict[str, Any]]) -> None:
```

```

C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-274-    """Write a resolved manifest JSON atomically
enough for local tooling."""
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-275-
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-276-    out = Path(path)
--
backend\eval\golden_manifest.py-279-    with tmp.open("w", encoding="utf-8") as f:
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-280-        json.dump(list(specs), f, indent=2)
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-281-        f.write("\n")
backend\eval\golden_manifest.py:282:    os.replace(tmp, out)
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-283-
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-284-
backend\eval\golden_manifest.py-285-def build_parser() -> argparse.ArgumentParser:
--
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-205-    attribution does not cure
provenance/licensing; this sits BEHIND counsel sign-off (P2) and the
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-206-    owner-asserted Fork-A / minors / biometric-
exclusion flags.
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-207-
backend\eval\golden_real_fixtures.py:208:    The fixture is written into a temp dir and promoted
(`os.replace`) to the final slug dir ONLY
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-209-    after the privacy scan passes ? a refused /
failed freeze leaves NO partial fixture dir
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-210-    (red-team #3).
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-211-
--
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-343-
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-344-    # --- (e) write into a TEMP dir, replay,
scan, and only THEN promote (red-team #3b) -----
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-345-    # Everything is written under a sibling
temp dir; the final slug dir is created by an atomic
backend\eval\golden_real_fixtures.py:346:    # os.replace ONLY after the privacy scan passes. A
refused/failed freeze therefore leaves NO
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-347-    # partial fixture dir behind. source_note
(if any) is scanned defensively but never written.
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-348-    fdir = fixtures_dir / out_slug
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-349-    fixtures_dir.mkdir(parents=True,
exist_ok=True)
--
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-387-    )
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-
indexer\backend\eval\golden_real_fixtures.py-388-
backend\eval\golden_real_fixtures.py-389-    # --- (g) PROMOTE atomically: temp slug dir ?
final slug dir (scan passed) -----
backend\eval\golden_real_fixtures.py:390:    os.replace(str(tmp_dir), str(fdir))
backend\eval\golden_real_fixtures.py-391-    finally:

```

```
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-  
indexer\backend\eval\golden_real_fixtures.py-392-          # Clean up the temp parent whether we  
promoted or refused (a refusal leaves NO partial dir).  
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-  
indexer\backend\eval\golden_real_fixtures.py-393-          shutil.rmtree(str(tmp_parent),  
ignore_errors=True)
```

22. assistant (2026-07-03T17:34:21.963Z)

Now let me search for manifest loading and env-var parsing patterns:

23. user (2026-07-03T17:34:23.119Z)

```
36-     2.0,  
37-     2.0,  
38-) # the WasbConfig / trajectory_to_action_windows defaults  
39-  
40-  
41:def load_golden_gts(csv_path: str) -> List[Interval]:  
42-    """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.  
43-  
44-    Accepts BOTH golden conventions now (start_time/end_time AND the collector export's  
45-    start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this  
46-    driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.  
47-  
48-  
49-    labels_csv: str,  
50-    fps: float,  
51-    frame_width: float,  
52-    iou: float = 0.5,  
53-) -> dict:  
54-    gts = load_golden_gts(labels_csv)  
55-    if not gts:  
56-        raise SystemExit(f"no usable golden rallies in {labels_csv}")  
57-    predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)  
58-    grid = default_grid()  
59-  
60-  
61-  
62-  
63-  
64-  
65-  
66-  
67-  
68-  
69-  
70-  
71-  
72-  
73-  
74-  
75-  
76-  
77-  
78-  
79-  
80-  
81-  
82-  
83-  
84-  
85-  
86-  
87-  
88-  
89-  
90-  
91-  
92-  
93-  
94-  
95-  
96-  
97-  
98-  
99-  
100-  
101-  
102-  
103-  
104-  
105-  
106-  
107-  
108-  
109-  
110-  
111-  
112-  
113-  
114-  
115-  
116-  
117-  
118-  
119-  
120-  
121-  
122-  
123-  
124-  
125-  
126-  
127-  
128-  
129-  
130-  
131-  
132-  
133-  
134-  
135-  
136-  
137-  
138-  
139-  
140-  
141-  
142-  
143-  
144-  
145-  
146-  
147-  
148-  
149-  
150-  
151-  
152-  
153-  
154-  
155-  
156-  
157-  
158-  
159-  
160-  
161-  
162-  
163-  
164-  
165-  
166-  
167-  
168-  
169-  
170-  
171-  
172-  
173-  
174-  
175-  
176-  
177-  
178-  
179-  
180-  
181-  
182-  
183-  
184-  
185-  
186-  
187-  
188-  
189-  
190-  
191-  
192-  
193-  
194-  
195-  
196-  
197-  
198-  
199-  
200-  
201-  
202-  
203-  
204-  
205-  
206-  
207-  
208-  
209-  
210-  
211-  
212-  
213-  
214-  
215-  
216-  
217-  
218-  
219-  
220-  
221-  
222-  
223-  
224-  
225-  
226-  
227-  
228-  
229-  
230-  
231-  
232-  
233-  
234-  
235-  
236-  
237-  
238-  
239-  
240-  
241-  
242-  
243-  
244-  
245-  
246-  
247-  
248-  
249-  
250-  
251-  
252-  
253-  
254-  
255-  
256-  
257-  
258-  
259-  
260-  
261-  
262-  
263-  
264-  
265-  
266-  
267-  
268-  
269-  
270-  
271-  
272-  
273-  
274-  
275-  
276-  
277-  
278-  
279-  
280-  
281-  
282-  
283-  
284-  
285-  
286-  
287-  
288-  
289-  
290-  
291-  
292-  
293-  
294-  
295-  
296-  
297-  
298-  
299-  
300-  
301-  
302-  
303-  
304-  
305-  
306-  
307-  
308-  
309-  
310-  
311-  
312-  
313-  
314-  
315-  
316-  
317-  
318-  
319-  
320-  
321-  
322-  
323-  
324-  
325-  
326-  
327-  
328-  
329-  
330-  
331-  
332-  
333-  
334-  
335-  
336-  
337-  
338-  
339-  
340-  
341-  
342-  
343-  
344-  
345-  
346-  
347-  
348-  
349-  
350-  
351-  
352-  
353-  
354-  
355-  
356-  
357-  
358-  
359-  
360-  
361-  
362-  
363-  
364-  
365-  
366-  
367-  
368-  
369-  
370-  
371-  
372-  
373-  
374-  
375-  
376-  
377-  
378-  
379-  
380-  
381-  
382-  
383-  
384-  
385-  
386-  
387-  
388-  
389-  
390-  
391-  
392-  
393-  
394-  
395-  
396-  
397-  
398-  
399-  
400-  
401-  
402-  
403-  
404-  
405-  
406-  
407-  
408-  
409-  
410-  
411-  
412-  
413-  
414-  
415-  
416-  
417-  
418-  
419-  
420-  
421-  
422-  
423-  
424-  
425-  
426-  
427-  
428-  
429-  
430-  
431-  
432-  
433-  
434-  
435-  
436-  
437-  
438-  
439-  
440-  
441-  
442-  
443-  
444-  
445-  
446-  
447-  
448-  
449-  
450-  
451-  
452-  
453-  
454-  
455-  
456-  
457-  
458-  
459-  
460-  
461-  
462-  
463-  
464-  
465-  
466-  
467-  
468-  
469-  
470-  
471-  
472-  
473-  
474-  
475-  
476-  
477-  
478-  
479-  
480-  
481-  
482-  
483-  
484-  
485-  
486-  
487-  
488-  
489-  
490-  
491-  
492-  
493-  
494-  
495-  
496-  
497-  
498-  
499-  
500-  
501-  
502-  
503-  
504-  
505-  
506-  
507-  
508-  
509-  
510-  
511-  
512-  
513-  
514-  
515-  
516-  
517-  
518-  
519-  
520-  
521-  
522-  
523-  
524-  
525-  
526-  
527-  
528-  
529-  
530-  
531-  
532-  
533-  
534-  
535-  
536-  
537-  
538-  
539-  
540-  
541-  
542-  
543-  
544-  
545-  
546-  
547-  
548-  
549-  
550-  
551-  
552-  
553-  
554-  
555-  
556-  
557-  
558-  
559-  
560-  
561-  
562-  
563-  
564-  
565-  
566-  
567-  
568-  
569-  
570-  
571-  
572-  
573-  
574-  
575-  
576-  
577-  
578-  
579-  
580-  
581-  
582-  
583-  
584-  
585-  
586-  
587-  
588-  
589-  
590-  
591-  
592-  
593-  
594-  
595-  
596-  
597-  
598-  
599-  
600-  
601-  
602-  
603-  
604-  
605-  
606-  
607-  
608-  
609-  
610-  
611-  
612-  
613-  
614-  
615-  
616-  
617-  
618-  
619-  
620-  
621-  
622-  
623-  
624-  
625-  
626-  
627-  
628-  
629-  
630-  
631-  
632-  
633-  
634-  
635-  
636-  
637-  
638-  
639-  
640-  
641-  
642-  
643-  
644-  
645-  
646-  
647-  
648-  
649-  
650-  
651-  
652-  
653-  
654-  
655-  
656-  
657-  
658-  
659-  
660-  
661-  
662-  
663-  
664-  
665-  
666-  
667-  
668-  
669-  
670-  
671-  
672-  
673-  
674-  
675-  
676-  
677-  
678-  
679-  
680-  
681-  
682-  
683-  
684-  
685-  
686-  
687-  
688-  
689-  
690-  
691-  
692-  
693-  
694-  
695-  
696-  
697-  
698-  
699-  
700-  
701-  
702-  
703-  
704-  
705-  
706-  
707-  
708-  
709-  
710-  
711-  
712-  
713-  
714-  
715-  
716-  
717-  
718-  
719-  
720-  
721-  
722-  
723-  
724-  
725-  
726-  
727-  
728-  
729-  
730-  
731-  
732-  
733-  
734-  
735-  
736-  
737-  
738-  
739-  
740-  
741-  
742-  
743-  
744-  
745-  
746-  
747-  
748-  
749-  
750-  
751-  
752-  
753-  
754-  
755-  
756-  
757-  
758-  
759-  
760-  
761-  
762-  
763-  
764-  
765-  
766-  
767-  
768-  
769-  
770-  
771-  
772-  
773-  
774-  
775-  
776-  
777-  
778-  
779-  
780-  
781-  
782-  
783-  
784-  
785-  
786-  
787-  
788-  
789-  
790-  
791-  
792-  
793-  
794-  
795-  
796-  
797-  
798-  
799-  
800-  
801-  
802-  
803-  
804-  
805-  
806-  
807-  
808-  
809-  
810-  
811-  
812-  
813-  
814-  
815-  
816-  
817-  
818-  
819-  
820-  
821-  
822-  
823-  
824-  
825-  
826-  
827-  
828-  
829-  
830-  
831-  
832-  
833-  
834-  
835-  
836-  
837-  
838-  
839-  
840-  
841-  
842-  
843-  
844-  
845-  
846-  
847-  
848-  
849-  
850-  
851-  
852-  
853-  
854-  
855-  
856-  
857-  
858-  
859-  
860-  
861-  
862-  
863-  
864-  
865-  
866-  
867-  
868-  
869-  
870-  
871-  
872-  
873-  
874-  
875-  
876-  
877-  
878-  
879-  
880-  
881-  
882-  
883-  
884-  
885-  
886-  
887-  
888-  
889-  
890-  
891-  
892-  
893-  
894-  
895-  
896-  
897-  
898-  
899-  
900-  
901-  
902-  
903-  
904-  
905-  
906-  
907-  
908-  
909-  
910-  
911-  
912-  
913-  
914-  
915-  
916-  
917-  
918-  
919-  
920-  
921-  
922-  
923-  
924-  
925-  
926-  
927-  
928-  
929-  
930-  
931-  
932-  
933-  
934-  
935-  
936-  
937-  
938-  
939-  
940-  
941-  
942-  
943-  
944-  
945-  
946-  
947-  
948-  
949-  
950-  
951-  
952-  
953-  
954-  
955-  
956-  
957-  
958-  
959-  
960-  
961-  
962-  
963-  
964-  
965-  
966-  
967-  
968-  
969-  
970-  
971-  
972-  
973-  
974-  
975-  
976-  
977-  
978-  
979-  
980-  
981-  
982-  
983-  
984-  
985-  
986-  
987-  
988-  
989-  
990-  
991-  
992-  
993-  
994-  
995-  
996-  
997-  
998-  
999-  
1000-  
1001-  
1002-  
1003-  
1004-  
1005-  
1006-  
1007-  
1008-  
1009-  
1010-  
1011-  
1012-  
1013-  
1014-  
1015-  
1016-  
1017-  
1018-  
1019-  
1020-  
1021-  
1022-  
1023-  
1024-  
1025-  
1026-  
1027-  
1028-  
1029-  
1030-  
1031-  
1032-  
1033-  
1034-  
1035-  
1036-  
1037-  
1038-  
1039-  
1040-  
1041-  
1042-  
1043-  
1044-  
1045-  
1046-  
1047-  
1048-  
1049-  
1050-  
1051-  
1052-  
1053-  
1054-  
1055-  
1056-  
1057-  
1058-  
1059-  
1060-  
1061-  
1062-  
1063-  
1064-  
1065-  
1066-  
1067-  
1068-  
1069-  
1070-  
1071-  
1072-  
1073-  
1074-  
1075-  
1076-  
1077-  
1078-  
1079-  
1080-  
1081-  
1082-  
1083-  
1084-  
1085-  
1086-  
1087-  
1088-  
1089-  
1090-  
1091-  
1092-  
1093-  
1094-  
1095-  
1096-  
1097-  
1098-  
1099-  
1100-  
1101-  
1102-  
1103-  
1104-  
1105-  
1106-  
1107-  
1108-  
1109-  
1110-  
1111-  
1112-  
1113-  
1114-  
1115-  
1116-  
1117-  
1118-  
1119-  
1120-  
1121-  
1122-  
1123-  
1124-  
1125-  
1126-  
1127-  
1128-  
1129-  
1130-  
1131-  
1132-  
1133-  
1134-  
1135-  
1136-  
1137-  
1138-  
1139-  
1140-  
1141-  
1142-  
1143-  
1144-  
1145-  
1146-  
1147-  
1148-  
1149-  
1150-  
1151-  
1152-  
1153-  
1154-  
1155-  
1156-  
1157-  
1158-  
1159-  
1160-  
1161-  
1162-  
1163-  
1164-  
1165-  
1166-  
1167-  
1168-  
1169-  
1170-  
1171-  
1172-  
1173-  
1174-  
1175-  
1176-  
1177-  
1178-  
1179-  
1180-  
1181-  
1182-  
1183-  
1184-  
1185-  
1186-  
1187-  
1188-  
1189-  
1190-  
1191-  
1192-  
1193-  
1194-  
1195-  
1196-  
1197-  
1198-  
1199-  
1200-  
1201-  
1202-  
1203-  
1204-  
1205-  
1206-  
1207-  
1208-  
1209-  
1210-  
1211-  
1212-  
1213-  
1214-  
1215-  
1216-  
1217-  
1218-  
1219-  
1220-  
1221-  
1222-  
1223-  
1224-  
1225-  
1226-  
1227-  
1228-  
1229-  
1230-  
1231-  
1232-  
1233-  
1234-  
1235-  
1236-  
1237-  
1238-  
1239-  
1240-  
1241-  
1242-  
1243-  
1244-  
1245-  
1246-  
1247-  
1248-  
1249-  
1250-  
1251-  
1252-  
1253-  
1254-  
1255-  
1256-  
1257-  
1258-  
1259-  
1260-  
1261-  
1262-  
1263-  
1264-  
1265-  
1266-  
1267-  
1268-  
1269-  
1270-  
1271-  
1272-  
1273-  
1274-  
1275-  
1276-  
1277-  
1278-  
1279-  
1280-  
1281-  
1282-  
1283-  
1284-  
1285-  
1286-  
1287-  
1288-  
1289-  
1290-  
1291-  
1292-  
1293-  
1294-  
1295-  
1296-  
1297-  
1298-  
1299-  
1300-  
1301-  
1302-  
1303-  
1304-  
1305-  
1306-  
1307-  
1308-  
1309-  
1310-  
1311-  
1312-  
1313-  
1314-  
1315-  
1316-  
1317-  
1318-  
1319-  
1320-  
1321-  
1322-  
1323-  
1324-  
1325-  
1326-  
1327-  
1328-  
1329-  
1330-  
1331-  
1332-  
1333-  
1334-  
1335-  
1336-  
1337-  
1338-  
1339-  
1340-  
1341-  
1342-  
1343-  
1344-  
1345-  
1346-  
1347-  
1348-  
1349-  
1350-  
1351-  
1352-  
1353-  
1354-  
1355-  
1356-  
1357-  
1358-  
1359-  
1360-  
1361-  
1362-  
1363-  
1364-  
1365-  
1366-  
1367-  
1368-  
1369-  
1370-  
1371-  
1372-  
1373-  
1374-  
1375-  
1376-  
1377-  
1378-  
1379-  
1380-  
1381-  
1382-  
1383-  
1384-  
1385-  
1386-  
1387-  
1388-  
1389-  
1390-  
1391-  
1392-  
1393-  
1394-  
1395-  
1396-  
1397-  
1398-  
1399-  
1400-  
1401-  
1402-  
1403-  
1404-  
1405-  
1406-  
1407-  
1408-  
1409-  
1410-  
1411-  
1412-  
1413-  
1414-  
1415-  
1416-  
1417-  
1418-  
1419-  
1420-  
1421-  
1422-  
1423-  
1424-  
1425-  
1426-  
1427-  
1428-  
1429-  
1430-  
1431-  
1432-  
1433-  
1434-  
1435-  
1436-  
1437-  
1438-  
1439-  
1440-  
1441-  
1442-  
1443-  
1444-  
1445-  
1446-  
1447-  
1448-  
1449-  
1450-  
1451-  
1452-  
1453-  
1454-  
1455-  
1456-  
1457-  
1458-  
1459-  
1460-  
1461-  
1462-  
1463-  
1464-  
1465-  
1466-  
1467-  
1468-  
1469-  
1470-  
1471-  
1472-  
1473-  
1474-  
1475-  
1476-  
1477-  
1478-  
1479-  
1480-  
1481-  
1482-  
1483-  
1484-  
1485-  
1486-  
1487-  
1488-  
1489-  
1490-  
1491-  
1492-  
1493-  
1494-  
1495-  
1496-  
1497-  
1498-  
1499-  
1500-  
1501-  
1502-  
1503-  
1504-  
1505-  
1506-  
1507-  
1508-  
1509-  
1510-  
1511-  
1512-  
1513-  
1514-  
1515-  
1516-  
1517-  
1518-  
1519-  
1520-  
1521-  
1522-  
1523-  
1524-  
1525-  
1526-  
1527-  
1528-  
1529-  
1530-  
1531-  
1532-  
1533-  
1534-  
1535-  
1536-  
1537-  
1538-  
1539-  
1540-  
1541-  
1542-  
1543-  
1544-  
1545-  
1546-  
1547-  
1548-  
1549-  
1550-  
1551-  
1552-  
1553-  
1554-  
1555-  
1556-  
1557-  
1558-  
1559-  
1560-  
1561-  
1562-  
1563-  
1564-  
1565-  
1566-  
1567-  
1568-  
1569-  
1570-  
1571-  
1572-  
1573-  
1574-  
1575-  
1576-  
1577-  
1578-  
1579-  
1580-  
1581-  
1582-  
1583-  
1584-  
1585-  
1586-  
1587-  
1588-  
1589-  
1590-  
1591-  
1592-  
1593-  
1594-  
1595-  
1596-  
1597-  
1598-  
1599-  
1600-  
1601-  
1602-  
1603-  
1604-  
1605-  
1606-  
1607-  
1608-  
1609-  
1610-  
1611-  
1612-  
1613-  
1614-  
1615-  
1616-  
1617-  
1618-  
1619-  
1620-  
1621-  
1622-  
1623-  
1624-  
1625-  
1626-  
1627-  
1628-  
1629-  
1630-  
1631-  
1632-  
1633-  
1634-  
1635-  
1636-  
1637-  
1638-  
1639-  
1640-  
1641-  
1642-  
1643-  
16
```

```

help="hit-detector sensitivity")
--
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-24-from typing import Any, Iterable, Sequence
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-25-
backend\eval\golden_manifest.py:26:DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-27-SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-28-DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
--
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-287-          description="Check/rebase the local golden
corpus manifest paths."
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-288-      )
backend\eval\golden_manifest.py:289:      p.add_argument("--manifest", default=DEFAULT_MANIFEST)
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-290-      p.add_argument(
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\golden_manifest.py-291-          "--repo-root",
--
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\fusion_golden.py-10-
backend\eval\fusion_golden.py-11-Run (on the GPU box):
backend\eval\fusion_golden.py:12:      python -m backend.eval.fusion_golden --manifest
output/human_lovo_manifest.json --out-dir output
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\fusion_golden.py-13-
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\fusion_golden.py-14-Each output `output/<name>_golden_features.json` is the
replayable feature substrate ? re-run
--
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\fusion_golden.py-235-          description="Extract golden-set shuttle+person
features (fusion P3 GPU step)."
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\fusion_golden.py-236-      )
backend\eval\fusion_golden.py:237:      ap.add_argument("--manifest",
default="output/human_lovo_manifest.json")
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\fusion_golden.py-238-      ap.add_argument("--out-dir", default="output")
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\fusion_golden.py-239-      ap.add_argument("--frame-skip", type=int,
default=3)
--
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-13-diff two windowings with a paired sign-test + CI (the
per-tier ``delta``).
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-14-
backend\eval\rally_seg_eval.py:15:GROUND TRUTH: the golden manifest
(``output/human_lovo_manifest.json``) gives each video's
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-16-trajectory path + fps + frame_width; the GT rally
intervals come from
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-17-``output/<name>_golden_features.json`` (the ``gts``
key) so this needs no extra label files.
--
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-32-from backend.eval.windowing import PRESETS, SERVED,
WindowingPreset
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-

```

```

indexer\backend\eval\rally_seg_eval.py-33-
backend\eval\rally_seg_eval.py:34:DEFAULT_MANIFEST = os.path.join("output",
"human_lovo_manifest.json")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-35-DEFAULT_FEATURES_DIR = "output"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-36-DEFAULT_IQU = 0.5
--
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-46-
backend\eval\rally_seg_eval.py-47-#: Multicourt golden clips ? the court-type lookup for the
stratified report. The golden manifest
backend\eval\rally_seg_eval.py:48-#: (``output/human_lovo_manifest.json``) is gitignored, so
this committed set is the AUDITABLE source
backend\eval\rally_seg_eval.py-49-#: of the single-vs-multicourt strata, kept in sync with
``docs/GOLDEN_VIDEOS.md`` (the clips tagged
backend\eval\rally_seg_eval.py-50-#: "multicourt"). Multicourt footage carries background games
on adjacent courts ? the dominant
--
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-183-#
----- #
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-184-def load_golden(
backend\eval\rally_seg_eval.py:185:    manifest_path: str = DEFAULT_MANIFEST, features_dir: str
= DEFAULT_FEATURES_DIR
backend\eval\rally_seg_eval.py-186-) -> List[Dict[str, Any]]:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-187-    """Load each golden video's trajectory path + fps
+ frame_width (manifest) and GT rally
--
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-626-        description="Rally-segmentation eval vs
golden labels (offline).")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-627-    )
backend\eval\rally_seg_eval.py:628:    ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-629-    ap.add_argument("--features-dir",
default=DEFAULT_FEATURES_DIR)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seg_eval.py-630-    ap.add_argument(
--
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seq_proto.py-24-
backend\eval\rally_seq_proto.py-25-Run:
backend\eval\rally_seq_proto.py:26:    python -m backend.eval.rally_seq_proto --manifest
output/human_lovo_manifest.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seq_proto.py-27-"""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\rally_seq_proto.py-28-
--
backend\eval\serve_contrast.py-21-CLI::
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\serve_contrast.py-22-
backend\eval\serve_contrast.py:23:    python -m backend.eval.serve_contrast --manifest
output/human_lovo_manifest.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\serve_contrast.py-24-"""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\serve_contrast.py-25-
--
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\serve_contrast.py-179-    ap.add_argument(

```

```

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\serve_contrast.py-180-      "--manifest",
backend\eval\serve_contrast.py:181:      default="output/human_lovo_manifest.json",
backend\eval\serve_contrast.py-182-      help="JSON: [{name, trajectory, fps, frame_width,
labels}, ...]",
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\serve_contrast.py-183-    )
--
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\served_gate_a.py-18-
backend\eval\served_gate_a.py-19-Run (owner box, golden trajectories present):
backend\eval\served_gate_a.py:20:  python -m backend.eval.served_gate_a --manifest
output/human_lovo_manifest.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\served_gate_a.py-21-""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\served_gate_a.py-22-
--
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\served_gate_a.py-206-      description="Verify served Gate-A on real
footage via the production FusionSegmenter path."
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\served_gate_a.py-207-    )
backend\eval\served_gate_a.py:208:  ap.add_argument("--manifest",
default="output/human_lovo_manifest.json")
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\served_gate_a.py-209-  ap.add_argument("--iou", type=float, default=0.5)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\backend\eval\served_gate_a.py-210-  args = ap.parse_args(argv)

```

26. assistant (2026-07-03T17:34:27.834Z)

Now let me search for env-var pathsep parsing patterns:

27. user (2026-07-03T17:34:28.689Z)

```

backend\eval\golden_manifest.py:198:  return [Path(p) for p in raw.split(os.pathsep) if
p.strip()]
backend\eval\golden_manifest.py:301:      f"{SEARCH_ROOTS_ENV} separated by os.pathsep."

```

28. assistant (2026-07-03T17:34:32.283Z)

Let me now check the specific diff sections that use load_resolved_manifest:

29. user (2026-07-03T17:34:32.724Z)

```

1      """One-command rally-segmentation eval ? score a windowing against the golden labels
with the
2      over-segmentation-sensitive metric set, honestly.
3
4      This is the harness the rally-quality project (`docs/RALLY_QUALITY_RESEARCH.md` $4.5 /
$7)
5      measures every tier against. It closes the loop the over-merge finding exposed: plain
temporal
6      IoU-F1 is blind to merged mega-windows, so we report it **alongside** segmental F1@k,
the
7      segment-count ratio, and the explicit ``merge_split`` breakdown
(``segmentation_metrics``).
8
9      For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds
candidate
10     windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the
SERVED
11     operating point and any future windowing the same way) ? scores the windows-as-rallies

```

```

against
12     the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI,
and can
13     diff two windowings with a paired sign-test + CI (the per-tier ``delta``).
14
15     GROUND TRUTH: the golden manifest (``output/human_lovo_manifest.json``) gives each
video's
16     trajectory path + fps + frame_width; the GT rally intervals come from
17     ``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label
files.
18
19     Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
20     ""
21
22     from __future__ import annotations
23
24     import argparse
25     import json
26     import os
27     from typing import Any, Dict, List, Optional, Tuple
28
29     from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
30     from backend.eval.golden_manifest import load_resolved_manifest
31     from backend.eval.metrics import Interval
32     from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
33
34     DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
35     DEFAULT_FEATURES_DIR = "output"
36     DEFAULT_IOU = 0.5
37
38     #: Long-rally lens threshold (s). A rally is "long" (an eye-catching highlight) when its
duration
39     #: (end ? start) exceeds this. The local detector over-fires, and its false positives
are mostly
40     #: SHORT (motion blips, background-court fragments on multicourt footage); filtering to
long rallies
41     #: strips most of those FPs and reveals the real product quality on the long-rally happy
path. Used
42     #: as the default split point for the stratified report (``--long-tau`` overrides it).
DURATION, not
43     #: shot-count, is the filter: golden ``shots_count`` is unlabeled and the no-AI path
reports it as
44     #: "Unknown", whereas duration is derived from the same start/end labels we already
trust.
45     LONG_RALLY_TAU = 5.0
46
47     #: Multicourt golden clips ? the court-type lookup for the stratified report. The golden
manifest
48     #: (``output/human_lovo_manifest.json``) is gitignored, so this committed set is the
AUDITABLE source
49     #: of the single-vs-multicourt strata, kept in sync with ``docs/GOLDEN_VIDEOS.md`` (the
clips tagged
50     #: "multicourt"). Multicourt footage carries background games on adjacent courts ? the
dominant
51     #: source of short false positives ? so we report it separately. A manifest entry MAY
carry an
52     #: explicit ``court_type`` field, which overrides this set (see :func:`court_type_for`).
53     MULTICOURT_CLIPS = frozenset(
54         {"mahadevpura_2", "GX010128", "Badminton_BXH_2", "Boxhill_Doubles"}
55     )
56
57
58     def court_type_for(video: Dict[str, Any]) -> str:
59         """"multicourt`` or ``single`` for a golden video. Honors an explicit manifest
60         ``court_type`` field when present, else falls back to membership in

```

```

``MULTICOURT_CLIPS``
61         (keyed by ``name``), else ``'single'``. """
62         ct = video.get("court_type")
63         if isinstance(ct, str) and ct:
64             return ct
65         return "multicourt" if video.get("name") in MULTICOURT_CLIPS else "single"
66
67
68     def filter_by_min_duration(ivs: List[Interval], min_duration: float) -> List[Interval]:
69         """The long-rally lens: keep only intervals strictly longer than ``min_duration``
seconds.
70
71         ``min_duration <= 0`` is an EXACT passthrough (the default-OFF semantics ? every
real rally has
72         positive duration, so nothing is dropped). It's a FILTER, not a new metric: applied
identically
73         to predictions AND ground truth before any matching, so the tIoU and R2 paths stay
consistent. """
74         if min_duration <= 0.0:
75             return list(ivs)
76         return [iv for iv in ivs if (iv[1] - iv[0]) > min_duration]
77
78
79     # ----- #
80     # Pure scoring core (no I/O ? unit-testable)
81     # ----- #
82     def score_intervals(
83         preds: List[Interval],
84         gts: List[Interval],
85         iou: float = DEFAULT_IOU,
86         tau_start: Optional[float] = None,
87         tau_end: Optional[float] = None,
88         min_duration: float = 0.0,
89     ) -> Dict[str, Any]:
90         """The full per-video metric row for predicted vs ground-truth rally intervals.
91
92         Pairs temporal-IoU P/R/F1 + boundary error (``metrics.score``) with the
93         over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
so a
94         merged mega-window is *visible* even when IoU-F1 isn't moved.
95
96         Also carries the **R2** task-faithful block (``tolerance_metrics``, the headline
going forward;
97         tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
strict-1:1
98         tolerance-match ``tol_f1/tol_precision/tol_recall`` (over-production costs
precision) + the
99         start/end boundary-error medians + the over-seg guard counts. See
docs/R2_EVAL_METRIC_DESIGN.md.
100
101         ``min_duration`` > 0 applies the **long-rally lens**
(:func:`filter_by_min_duration`): BOTH preds
102         and GTs are filtered to rallies longer than that many seconds BEFORE any matching,
so the whole
103         row (tIoU set AND R2 block) scores only long rallies. Default 0.0 = OFF (exact
passthrough).
104         """
105         from backend.eval import tolerance_metrics as tm
106
107         preds = filter_by_min_duration(preds, min_duration)
108         gts = filter_by_min_duration(gts, min_duration)
109         ts = tm.TAU_START if tau_start is None else tau_start
110         te = tm.TAU_END if tau_end is None else tau_end
111         sc = metrics.score(preds, gts, iou_threshold=iou)
112         f1k = segmentation_metrics.f1_at_overlaps(preds, gts)

```



```

113     ms = segmentation_metrics.merge_split_report(preds, gts)
114     tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
115     be = tm.boundary_error_report(
116         preds, gts
117     ) # unconditional best-overlap (the R4-aiming diagnostic)
118     osr = tm.over_seg_report(preds, gts)
119     return {
120         "f1": sc.f1,
121         "precision": sc.precision,
122         "recall": sc.recall,
123         "mean_iou": sc.mean_iou,
124         "mean_start_error": sc.mean_start_error,
125         "mean_end_error": sc.mean_end_error,
126         "f1@0.1": f1k[0.1],
127         "f1@0.25": f1k[0.25],
128         "f1@0.5": f1k[0.5],
129         "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
130         "merge_rate": ms["merge_rate"],
131         "split_rate": ms["split_rate"],
132         "merges": ms["merges"],
133         "splits": ms["splits"],
134         "missed": ms["missed"],
135         "spurious": ms["spurious"],
136         "num_pred": len(preds),
137         "num_gt": len(gts),
138         # --- R2 (tolerance-match) ? the task-faithful headline block ---
139         "tau_start": ts,
140         "tau_end": te,
141         "tol_f1": tol_f1,
142         "tol_precision": tol_p,
143         "tol_recall": tol_r,
144         "tol_matched": float(len(tol_matches)),
145         "dstart_abs_median": be["dstart"]["abs_median"],
146         "dend_abs_median": be["dend"]["abs_median"],
147         "split_count": osr["split_count"],
148         "merge_count": osr["merge_count"],
149     }
150
151
152     _AGG_KEYS = (
153         "f1",
154         "f1@0.25",
155         "f1@0.5",
156         "merge_rate",
157         "split_rate",
158         "segment_count_ratio",
159         "precision",
160         "recall",
161         "tol_f1",
162         "tol_precision",
163         "tol_recall",
164         "dstart_abs_median",
165         "dend_abs_median",
166     )
167
168
169     def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
170         """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
171         across videos.
172
173         Per-video means (not pooled) keep videos the unit of evidence, matching the
174         harness's
175
176         leave-one-video-out discipline (windows within a video are correlated)."""
177         out: Dict[str, Any] = {"n": len(rows)}
178         for k in _AGG_KEYS:

```

```

176         vals = [r[k] for r in rows]
177         out[k] = eval_stats.mean_with_ci(vals) if vals else None
178     return out
179
180
181     # ----- #
182     # Golden corpus I/O
183     # ----- #
184     def load_golden(
185         manifest_path: str = DEFAULT_MANIFEST, features_dir: str = DEFAULT_FEATURES_DIR
186     ) -> List[Dict[str, Any]]:
187         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
188         intervals (``<name>_golden_features.json`` ``gts``). Videos with no GTs are kept but
flagged."""
189         manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
190         out: List[Dict[str, Any]] = []
191         for v in manifest:
192             feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
193             gts: List[Interval] = []
194             if os.path.exists(feat):
195                 with open(feat) as fh:
196                     gts = [
197                         (float(s), float(e)) for s, e in (json.load(fh).get("gts") or [])
198                     ]
199             entry: Dict[str, Any] = {
200                 "name": v["name"],

```

30. assistant (2026-07-03T17:34:34.899Z)

Now let me check what `calibrate_wasb` defines for manifest loading:

31. user (2026-07-03T17:34:35.621Z)

```

1     """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
2
3     Wires the already-tested pieces together:
4     WASB trajectory CSV (Frame,Visibility,X,Y)
5     -> TrackNetRunner.trajectory_to_action_windows(cfg) [predict_fn]
6     -> backend.eval.calibration (improvement_report / cross_validate)
7
8     Run (after a full-video WASB pass exists):
9     python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10         --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12     Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13     same GT is an **optimistic** upper bound. The trustworthy number is the
14     cross-validated held-out **recall** (and, once a 2nd labelled video exists,
15     `leave_one_video_out` for precision/F1 generalization).
16     """
17
18     from __future__ import annotations
19
20     import argparse
21     from typing import Callable, List, Tuple
22
23     from backend.eval.calibration import (
24         cross_validate,
25         evaluate,
26         improvement_report,
27         select_best,
28     )
29     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30

```

```

31 Interval = Tuple[float, float]
32 Config = Tuple[float, float, float] # (velocity_thresh, merge_gap, min_window_duration)
33
34 BASELINE: Config = (
35     0.02,
36     2.0,
37     2.0,
38 ) # the WasbConfig / trajectory_to_action_windows defaults
39
40
41 def load_golden_gts(csv_path: str) -> List[Interval]:
42     """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
43
44     Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
45     start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
46     driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
47     """
48     from backend.eval.gt_loader import load_gt_intervals
49
50     return load_gt_intervals(csv_path, strict=False)
51
52
53 def make_predict_fn(
54     trajectory_csv: str, fps: float, frame_width: float
55 ) -> Callable[[Config], List[Interval]]:
56     """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
57     points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
58
59     def predict_fn(cfg: Config) -> List[Interval]:
60         vt, mg, mwd = cfg
61         wins = TrackNetRunner.trajectory_to_action_windows(
62             points,
63             fps=fps,
64             frame_width=frame_width,
65             velocity_thresh=vt,
66             merge_gap=mg,
67             min_window_duration=mwd,
68         )
69         return [(w["start"], w["end"]) for w in wins]
70
71     return predict_fn
72
73
74 def default_grid() -> List[Config]:
75     return [
76         (vt, mg, mwd)
77         for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
78         for mg in (1.0, 2.0, 3.0)
79         for mwd in (1.0, 2.0, 3.0)
80     ]
81
82
83 def run(
84     trajectory_csv: str,
85     labels_csv: str,
86     fps: float,
87     frame_width: float,
88     iou: float = 0.5,
89 ) -> dict:
90     gts = load_golden_gts(labels_csv)
91     if not gts:
92         raise SystemExit(f"no usable golden rallies in {labels_csv}")
93     predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
94     grid = default_grid()
95

```

```

96         base = evaluate(predict_fn(BASELINE), gts, iou)
97         best_cfg, best_f1 = select_best(
98             predict_fn, grid, gts, metric="f1", iou_threshold=iou
99         )
100         fit = improvement_report(

```

32. assistant (2026-07-03T17:34:38.052Z)

Now let me check if there's a shared config helper or if there are duplicate CLI defaults:

33. user (2026-07-03T17:34:39.900Z)

```

backend/eval/golden_manifest.py:DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
backend/eval/golden_manifest.py:    p.add_argument("--manifest", default=DEFAULT_MANIFEST)
backend/eval/rally_seg_eval.py:DEFAULT_MANIFEST = os.path.join("output",
"human_lovo_manifest.json")
backend/eval/rally_seg_eval.py:    manifest_path: str = DEFAULT_MANIFEST, features_dir: str =
DEFAULT_FEATURES_DIR
backend/eval/rally_seg_eval.py:    ap.add_argument("--manifest", default=DEFAULT_MANIFEST)

```

34. assistant (2026-07-03T17:34:41.889Z)

Perfect! Let me check for duplicated manifest defaults across all eval files:

35. user (2026-07-03T17:34:42.951Z)

```

backend/eval/fusion_audio.py:129:    ap.add_argument("--manifest",
default="output/human_lovo_manifest.json")
backend/eval/fusion_golden.py:237:    ap.add_argument("--manifest",
default="output/human_lovo_manifest.json")
backend/eval/golden_manifest.py:26:DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
backend/eval/serve_contrast.py:181:    default="output/human_lovo_manifest.json",
backend/eval/served_gate_a.py:208:    ap.add_argument("--manifest",
default="output/human_lovo_manifest.json")

```

36. assistant (2026-07-03T17:34:45.584Z)

Now let me check if any of these modules use config helpers from backend/config/:

37. user (2026-07-03T17:34:46.649Z)

```

backend/eval/served_gate_a.py:from backend.config.models import FusionConfig, IndexingConfig,
WasbIndexConfig
backend/eval/windowing.py:from backend.config.models import IndexingConfig

```

38. assistant (2026-07-03T17:34:49.260Z)

Let me look at how other eval modules handle env-var splits:

39. user (2026-07-03T17:34:50.347Z)

```

backend/eval/gemini_refine.py:    api_key = os.environ.get("GEMINI_API_KEY")
backend/eval/gemini_refine.py:    api_key = os.environ.get("GEMINI_API_KEY")
backend/eval/golden_manifest.py:    raw = os.environ.get(SEARCH_ROOTS_ENV, "")
backend/eval/golden_manifest.py:    return [Path(p) for p in raw.split(os.pathsep) if p.strip()]
backend/eval/golden_manifest.py:        f"{SEARCH_ROOTS_ENV} separated by os.pathsep."

```

40. assistant (2026-07-03T17:34:52.670Z)

Now let me search for existing imports of json loading patterns in experiment.py to see if there's a shared schema:

41. user (2026-07-03T17:34:54.104Z)

```

31: - the promotion gate stays `run_regression.py` + `regression_baseline.json`
42:import json
68:DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
159:     blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
193:     ``*_golden_features.json`` row (``start``/``end``` + velocity stats, no cue dicts) for
clips
221: Reads every FULL-FUSION-SCHEMA ``*_golden_features.json`` in ``features_dir`` into a
244: for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
246:     d = json.load(f)
685: `regression_baseline.json`; deliberately the size of a baseline file, not a service
713:     entries = json.load(f)
717:     json.dump(entries, f, indent=2)

```

42. assistant (2026-07-03T17:34:56.047Z)

Let me check the gt_loader module to see how manifest specs are loaded:

43. user (2026-07-03T17:34:56.564Z)

```

1      """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3      The repo historically carried two incompatible loaders:
4      - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5        ``curate export`` format), raises on bad rows;
6      - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7        (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9      A file in one convention fed to the other loader yielded zero/garbage intervals with no
10     error. This module accepts BOTH conventions (header-mapped, else positional), so every
11     quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12     flows through one reader. Both legacy entry points now delegate here.
13
14     Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15     - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16       columns ? rally_number, ending_reason, sport, ?? are ignored);
17     - no header: columns 0,1 positionally.
18     Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19     (``annotations.parse_timestamp``).
20
21     """
22
23     import csv
24     import logging
25     from typing import List, Optional, Tuple
26
27     from backend.eval.annotations import parse_timestamp
28
29     logger = logging.getLogger(__name__)
30
31     Interval = Tuple[float, float]
32
33     _START_NAMES = ("start", "start_time")
34     _END_NAMES = ("end", "end_time")
35
36     def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
37         """If `row` is a header naming start/end columns, return their indices, else
38         None."""
39         cells = [(c or "").strip().lower() for c in row]
40         start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
41         end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
42         if start_idx is not None and end_idx is not None:
43             return start_idx, end_idx
44         return None

```

```

44
45 def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
46     """Load GT rally intervals from either golden-CSV convention.
47
48     strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
49     (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
50     strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
51     skipped ? a silently shrinking GT set flatters precision.
52
53     Returns intervals sorted by start; overlapping GT intervals are logged as a warning
54     (they break one-to-one matching and usually indicate a labeling error).
55     """
56     intervals: List[Interval] = []
57     skipped = 0
58     cols: Optional[Tuple[int, int]] = None # (start_idx, end_idx); None until detected
59     saw_header = False
60
61     with open(csv_path, newline="", encoding="utf-8-sig") as f:
62         for lineno, row in enumerate(csv.reader(f), start=1):
63             if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
64                 continue
65             if cols is None and not saw_header:
66                 header = _header_columns(row)
67                 if header is not None:
68                     cols = header
69                     saw_header = True
70                     continue # consume the header row
71             cols = (0, 1) # headerless: positional start,end
72             assert cols is not None # every branch above assigns it
73             si, ei = cols
74             try:
75                 if len(row) <= max(si, ei):
76                     raise ValueError(
77                         f"expected at least {max(si, ei) + 1} columns, got {row!r}"
78                     )
79                 start = parse_timestamp(row[si])
80                 end = parse_timestamp(row[ei])
81                 if start < 0:
82                     raise ValueError(f"negative start ({start})")
83                 if start >= end:
84                     raise ValueError(f"start ({start}) >= end ({end})")
85             except ValueError as e:
86                 if strict:
87                     raise ValueError(f"{csv_path}:{lineno}: {e}") from e
88                 skipped += 1
89                 continue
90             intervals.append((start, end))
91
92     if skipped:
93         logger.warning(
94             "%s: skipped %d malformed GT row(s) (strict=False)", csv_path, skipped
95         )
96
97     intervals.sort(key=lambda iv: iv[0])
98     for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
99         if s2 < e1:
100             logger.warning(

```

44. assistant (2026-07-03T17:34:58.701Z)

Now let me check if there are any config files that might define shared constants:

45. user (2026-07-03T17:35:00.800Z)

backend/config/loader.py

```
backend/config/models.py
backend/config/startup.py
backend/config/__init__.py
```

46. user (2026-07-03T17:35:04.112Z)

(Bash completed with no output)

47. assistant (2026-07-03T17:35:07.238Z)

Now let me look at specific evaluations to see what shared manifest loading pattern already exists or could exist:

48. user (2026-07-03T17:35:07.954Z)

```
1      """Distill Gemini (teacher) into the LOCAL-ONLY pipeline (student) ? calibrate the
2      on-device WASB+heuristic knobs to match Gemini's rally labels, so production runs
3      100% local (no cloud, privacy lever intact, ~zero marginal cost).
4
5      Pipeline being tuned (all local, no Gemini at runtime):
6          WASB trajectory CSV -> trajectory_to_action_windows(velocity, merge_gap, min_dur)
7                               -> rally_gate (min_crossings)                -> rally windows
8
9      Ground truth = Gemini full-context silver labels (one CSV per video). With >=2
10     videos we use leave-one-video-out (tune on the others, score the held-out one) ?
11     the honest "how precise is the local pipeline on an unseen video" number. The
12     output is a single local config + the precision it buys, with NO Gemini dependency
13     shipped.
14
15     Reuses: TrackNetRunner windowing, rally_gate, calibrate_wasb.load_golden_gts,
16     calibration.{select_best, leave_one_video_out, improvement_report}.
17
18     Run:
19         python -m backend.eval.calibrate_local --manifest videos.json
20     where videos.json = [{"name","trajectory","fps","frame_width","labels"}, ...]
21     (labels = a Gemini full-context CSV; trajectory = the WASB CSV).
22     """
23
24     from __future__ import annotations
25
26     import argparse
27     from typing import Callable, Dict, List, Tuple
28
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.calibration import (
31         evaluate,
32         improvement_report,
33         leave_one_video_out,
34         select_best,
35     )
36     from backend.eval.golden_manifest import load_resolved_manifest
37     from backend.pipeline.detectors import rally_gate
38     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
39
40     # (velocity_thresh, merge_gap, min_window_duration, min_crossings)
41     LocalConfig = Tuple[float, float, float, int]
42     BASELINE_LOCAL: LocalConfig = (
43         0.02,
44         2.0,
45         2.0,
46         0,
47     ) # today's local default (windowing, no gate)
48
49
```

```
50     def make_local_predict_fn(
```

49. assistant (2026-07-03T17:35:10.357Z)

Let me look for existing atomic write utilities in the codebase:

50. user (2026-07-03T17:35:11.708Z)

(Bash completed with no output)

51. user (2026-07-03T17:35:14.885Z)

```
    attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off (P2) and
the
    owner-asserted Fork-A / minors / biometric-exclusion flags.

The fixture is written into a temp dir and promoted (`os.replace`) to the final slug dir
ONLY
after the privacy scan passes ? a refused / failed freeze leaves NO partial fixture dir
(red-team #3).

Returns the recomputed ``expected.json`` snapshot dict (identical to ``replay_fixture``).
"""
# --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail ($4c / $7 ?) -----
reasons: List[str] = []
if minors_free is not True:
--

# --- (e) write into a TEMP dir, replay, scan, and only THEN promote (red-team #3b) -----
# Everything is written under a sibling temp dir; the final slug dir is created by an atomic
# os.replace ONLY after the privacy scan passes. A refused/failed freeze therefore leaves NO
# partial fixture dir behind. source_note (if any) is scanned defensively but never written.
fdir = fixtures_dir / out_slug
fixtures_dir.mkdir(parents=True, exist_ok=True)
if fdir.exists():
    raise ValueError(
        f"{fdir}: fixture dir already exists ? refuse to clobber (choose a new slug)"
    )
tmp_parent = Path(
--
    )

    # --- (g) PROMOTE atomically: temp slug dir ? final slug dir (scan passed) -----
    os.replace(str(tmp_dir), str(fdir))
finally:
    # Clean up the temp parent whether we promoted or refused (a refusal leaves NO partial
dir).
    shutil.rmtree(str(tmp_parent), ignore_errors=True)

# --- (h) append to the manifest with kind="cleared_real" (don't clobber synthetic) -----
_append_manifest_entry(prov, fixtures_dir)
return expected
```

52. assistant (2026-07-03T17:35:17.195Z)

Let me now check if there are any existing utilities that handle path resolution similar to what golden_manifest does:

53. user (2026-07-03T17:35:18.291Z)

```
27: def safe_output_filename(name: str) -> str:
45: def resolve_under_root(path: str, root: str) -> str:
58: def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
68:     resolve_under_root(path, allowed_root)
```


54. assistant (2026-07-03T17:35:21.970Z)

Let me verify repo_root pattern usage in other files:

55. user (2026-07-03T17:35:22.783Z)

```
1     import logging
2     import os
3     import subprocess
4     import tempfile
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
9         ffmpeg_bin,
10        ffmpeg_timeout_sec,
11        ffprobe_bin,
12        ffprobe_timeout_sec,
13        timeout_label,
14    )
15
16    logger = logging.getLogger(__name__)
17
18    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
19    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
20    # `font='Sans` fails). Overridable via stitching.watermark.font_path. This module lives
at
21    # backend/pipeline/stitcher/, so parents[2] == backend/.
22    _BUNDLED_FONT = (
23        Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
24    )
25
26
27    class VideoCompiler:
28        def __init__(
29            self,
30            output_dir: str,
31            end_padding: float = 1.0,
32            begin_padding: float = 0.5,
33            watermark: Optional[Any] = None,
34        ):
35            self.output_dir = output_dir
36            self.end_padding = end_padding
37            self.begin_padding = begin_padding
38            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
39            # normalized to a dict (or None) so this module stays config-package-agnostic.
40            self.watermark = self._normalize_watermark(watermark)
41            os.makedirs(output_dir, exist_ok=True)
42
43        @staticmethod
44        def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
45            if watermark is None:
46                return None
47            if hasattr(watermark, "model_dump"):
48                return watermark.model_dump()
49            if isinstance(watermark, dict):
50                return dict(watermark)
51            return None
52
53        @staticmethod
54        def _ff_quote(path: str) -> str:
55            """Make a filesystem path safe to embed inside single quotes in an ffmpeg
56            filtergraph operand (drawtext fontfile/textfile, movie= source): forward
```

```

57         slashes (Windows-friendly), escaped single quotes, AND escaped colons.
58
59         The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
60         option parser treats ':' as the option separator, so a Windows drive-letter
61         path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
62         the encode fails with "Invalid argument". This silently disabled the
63         on-by-default bundled-font watermark (and absolute logo paths) on Windows.""
64         return path.replace("\\", "/").replace("'", r"\').replace(":", r"\:")
65
66     @staticmethod
67     def _parse_time(val: Union[int, float, str]) -> float:
68         """Normalizes a timestamp value to float seconds.
69         Handles: float/int (pass-through), plain numeric strings ("12.4"),
70         and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
71         This mirrors the parse_time logic used in gemini.py to ensure
72         the stitcher can always consume whatever format the indexer produced.""
73         if isinstance(val, (int, float)):
74             return float(val)
75         if isinstance(val, str):
76             val = val.strip()
77             if ":" in val:
78                 # Handle MM:SS, HH:MM:SS, etc.
79                 parts = val.split(":")
80                 parts.reverse()
81                 total = 0.0
82                 for i, p in enumerate(parts):
83                     try:
84                         total += float(p) * (60**i)
85                     except ValueError:
86                         pass
87                 return total
88             try:
89                 return float(val)
90             except ValueError:
91                 logger.warning(
92                     f"Could not parse timestamp value '{val}' as a number. Defaulting to
93                     0.0."
94                 )
95                 return 0.0
96         logger.warning(
97             f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
98             Defaulting to 0.0."
99         )
100         return 0.0
101
102     @staticmethod

```

56. assistant (2026-07-03T17:35:25.238Z)

Let me check the watermark_i18n file:

57. user (2026-07-03T17:35:26.926Z)

```
23: _FONTS_DIR = Path(__file__).resolve().parents[2] / "assets" / "fonts"
```

58. assistant (2026-07-03T17:35:29.427Z)

Perfect! Now let me check the exact line in golden_fixtures.py that uses Path(__file__).resolve().parents:

59. user (2026-07-03T17:35:29.975Z)

```

70
71     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose

```

```

72     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
73     CURRENT_SCHEMA = 1
74
75     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
76     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
77     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
78
79     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
80     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
81     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
82     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
83     #: re-states these in its provenance; the replay reads them from there.
84     PINNED_VT = 0.02
85     PINNED_MERGE_GAP = 2.0
86     PINNED_MIN_WINDOW = 2.0
87     #: W1 (bounded-windowing) max-window seconds. 0 = OFF, matching the SERVED default.
NOTE: this is
88     #: only RECORDED (provenance/manifest) ? it is NOT applied by the replay.
``_pinned_proposal``
89     #: returns just ``(vt, mg, mwd)`` and ``distill_local.build_candidates`` (shared code,
deliberately
90     #: NOT modified by this harness) takes no max-window arg, so the fixtures characterize
the W1-OFF
91     #: path. Do NOT assume this value tracks a future SERVED W1 flip ? that would need new
wiring.
92     PINNED_MAX_WINDOW = 0.0
93
94     #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
stated
95     #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
exactly these.
96     SHUTTLE_FEATURES: Tuple[str, ...] = (
97         "duration",
98         "peak_velocity",
99         "mean_velocity",
100        "active_ratio",
101        "net_crossings",
102    )
103    assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
104        "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
105    )
106
107    #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
108    #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
109    FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
110
111    _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
112    _FLOAT_TOL = 1e-6 # abs tolerance when parse-comparing recomputed vs committed floats
113
114    #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
115    #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
116    _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
117
118    #: Strict slug charset for a real-fixture ``--label`` ? lowercase alnum, ``_``/``-`` (no
spaces,
119    #: no punctuation, no uppercase). A free-text label ("Lin Dan vs Lee, India Open 2024")

```

```

is an
120     #: attribution back-channel that ``_scan_forbidden`` does NOT catch (it isn't a
FORBIDDEN_KEY / MD5
121     #: / source-path token), so it is rejected up-front by construction (red-team: free-text
id leak).
122     _LABEL_SLUG_RE = re.compile(r"^[a-z0-9][a-z0-9_\-]*$")
123
124
125     # =====
126     # Synthetic trajectory generation (deterministic ? NO random)
127     # =====
128
129
130     @dataclass(frozen=True)
131     class RallySpec:
132         """One in-flight rally segment: the shuttle oscillates across the net midline.
133
134         ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
135         period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
136         net midline along the play axis. Deterministic sine motion ? fixed
137         velocity/crossings.
138         """
139         start_frame: int
140         n_frames: int
141         period: float = 30.0
142         amplitude: float = 400.0
143
144
145     @dataclass(frozen=True)
146     class GapSpec:
147         """Dead-time between rallies ? what separates one window from the next.
148
149         ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
150         ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
151         end an active run so the next rally forms a distinct window.
152         """
153
154         start_frame: int
155         n_frames: int
156         kind: str = "still" # "still" | "occluded"
157
158
159     @dataclass(frozen=True)

```

60. assistant (2026-07-03T17:35:44.611Z)

Excellent! Now I have all the information I need. Let me create a comprehensive analysis report:

Based on my thorough analysis of the PR #452 diff and the codebase, here are the concrete code reuse issues in the new `backend/eval/golden\_manifest.py` module:

```

```json
[
 {
 "file": "backend/eval/golden_manifest.py",
 "line": 151,
 "summary": "repo_root() duplicates Path(__file__).resolve().parents[2] pattern already used
in golden_fixtures.py:77 and compiler.py:23",
 "failure_scenario": "If the repo structure changes, both golden_manifest.repo_root() and
golden_fixtures.FIXTURES_DIR will need updates. The pattern should be centralized in a
backend.utils helper to avoid silent inconsistencies across modules that rely on this constant-
time computation."
 },
 {

```

```

 "file": "backend/eval/golden_manifest.py",
 "line": 26,
 "summary": "DEFAULT_MANIFEST='output/human_lovo_manifest.json' duplicates literal string
hardcoded in rally_seg_eval.py:34 (and spread across fusion_golden.py:237,
serve_contrast.py:181, served_gate_a.py:208, fusion_audio.py:129 with inline defaults)",
 "failure_scenario": "If the manifest path changes (e.g., renamed file, directory migration),
6+ modules require updates. rally_seg_eval already defines its own DEFAULT_MANIFEST
constant?golden_manifest should import/reuse it or place a shared constant in backend.config (or
a new backend.eval.manifest_config module) that all eval CLIs reference."
},
{
 "file": "backend/eval/golden_manifest.py",
 "line": 273,
 "summary": "write_manifest() implements atomic JSON write (tmp + os.replace) matching
golden_real_fixtures.py:390 pattern (temp ? promote) and golden_fixtures.py:571
_write_manifest",
 "failure_scenario": "No existing atomic-write utility; golden_manifest reimplements the
pattern inline. If JSON serialization options (indent, trailing newline, encoding) need to
change for consistency, 3+ locations require updates. Consider a backend.utils.atomic_json or
backend.eval.json_io helper."
},
{
 "file": "backend/eval/golden_manifest.py",
 "line": 198,
 "summary": "_env_roots() uses raw os.environ.get(key, '').split(os.pathsep) pattern. No
existing config helper found, but backend.utils.paths.py exists with helpers
(safe_output_filename, resolve_under_root, validate_input_path) that should own env/path parsing
utilities.",
 "failure_scenario": "RALLY_GOLDEN_SEARCH_ROOTS env-var splitting logic is custom per-module.
If a future module needs similar multi-path env-var parsing, duplication recurs.
backend.utils.paths or backend.config.loader should expose a env_path_list(key) helper."
},
{
 "file": "backend/eval/rally_seg_eval.py",
 "line": 34,
 "summary": "DEFAULT_MANIFEST constant in rally_seg_eval.py (line 34) and golden_manifest.py
(line 26) both define 'output/human_lovo_manifest.json' independently",
 "failure_scenario": "Mutation in one module does not sync to the other. When rally_seg_eval
refactored to use load_resolved_manifest (diff line 189), its own DEFAULT_MANIFEST constant was
not consolidated?now both modules own the literal, breaking DRY."
},
{
 "file": "backend/eval/golden_manifest.py",
 "line": 340,
 "summary": "_candidate_dirs() and _expand_root() implement candidate-path search (Annotation
Setup, Collect, output/) that mirrors the manual path-joining logic scattered in
backend/pipeline/stitcher/ and other modules using Path() joins",
 "failure_scenario": "No existing workspace-relative path resolver found.
backend.utils.workspace.py handles per-video workspaces (join_uri, resolve_workspace_root), but
not multi-candidate sibling-dir discovery. Path-rebasing logic may be re-needed when other
manifest types (annotation exports, feature catalogs) migrate to portable manifests."
}
]
...

```